

claude-windows / 144a31b7-7962-4fee-9d6f-8a7854daa63f

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\144a31b7-7962-4fee-9d6f-8a7854daa63f\subagents\workflows\wf_134043cd-8c3\agent-af5e7a5e74fbd57c1.jsonl`
- SHA-256: `6079efe2984d825f7a426edb4d661c94c5d434b8b2c3e5db7d62dc15b0db740e`
- Source modified: `2026-06-20T09:18:24+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf_134043cd-8c3`
- session_id: `144a31b7-7962-4fee-9d6f-8a7854daa63f`
```

Transcript

1. user (2026-06-20T09:14:02.099Z)

You are the lead architect consolidating three single-box-appliance designs into one recommendation + a tracked design-doc draft for KhelSutra's "locally installed, UI-powered rally indexer + reel generator".

RESEARCH (verified ground truth):

```
{
  "decoupled": {
    "storage_contract": "StorageBackend ABC = backend/storage/base.py:17. Abstract methods:
fetch_to_local(ref, dst=None, *, overwrite=False)->str (base.py:21), put(local_path, ref, *,
content_type=None)->StorageRef (:27), exists (:32), stat (:35),
list(prefix)->Iterator[StorageRef] (:38), delete (:42); plus a NON-abstract
signed_url(ref,*,ttl,method) that raises NotImplementedError by default (:45-49) ? callers MUST
treat that as expected and fall back to fetch_to_local (local can't sign; gdrive only
emulates).\n\nURI scheme (backend/storage/ref.py:48 parse_uri + docstring :1-13): bare path or
local:// => local (today's behaviour, IDENTITY passthrough, no copy); s3://<bucket>/<key> => s3
(covers AWS + Cloudflare R2 + DO Spaces + MinIO, distinguished only by endpoint_url); gcs:// (or
gs://, normalized) => gcs; gdrive://<folderId>/<path> => gdrive (follow-up stub). Windows
C:\\... has no :// so falls through to local. Value objects: StorageRef(backend,bucket,key,uri)
frozen dataclass with a .name property (ref.py:34-45);
StorageStat(size,modified,etag,content_type) (ref.py:25).\n\nThin URI helpers in
backend/storage/__init__.py: get_storage(backend,config,**overrides) (:43),
fetch(uri,dst,config,overwrite) (:55), put(local_path,uri,config,content_type) (:63),
list_uris(prefix_uri,config) (:71). Backends are lazy-resolved (__init__.py:26 _backend_class)
so `import backend.storage` never pulls boto3/google SDKs ? they import inside the concrete
module (s3.py:29 `import boto3`). CREDENTIALS NEVER IN CONFIG: s3.py:8-10 + S3Storage.__init__
(s3.py:22) takes only endpoint_url/region/addressing_style/signed_url_ttl; boto3's default chain
(env AWS_ACCESS_KEY_ID/SECRET, instance role, ~/.aws) supplies creds. A client can be injected
for tests (s3.py:26).\n\nLocalStorage (local.py:18): fetch_to_local is identity when dst is None
or dst==src (local.py:33), else shutil.copyfile; put is a copy that no-ops when src==dst
(:42-47); list walks the dir (:56). So backend='local' is byte-for-byte the pre-storage-layer
behaviour.\n\nBucket layout (04 doc §3, lines 56-66): videos/<md5>.mp4 (immutable, byte-
identical to what labels were made on), labels/<sport>/<video_id>.csv, manifest.json (the join
table ? needs an md5 field added, an M2 prereq per 02 §4), weights/<semver>/<model,manifest> +
weights/checkpoints/, outputs/<video_id>/<highlights.mp4, intervals.csv,
report.json>.\n\nStorageConfig (backend/config/models.py:291): backend:
Literal["local","s3","gcs","gdrive"]="local"; output_root:str=""; per-backend loose
dicts local/s3/gcs/gdrive (:301-304) passed straight to the backend constructor
(endpoints/regions only, no secrets).",
    "worker_cli": "Entrypoint backend/worker/__main__.py ? `python -m backend.worker
{infer|train}`. Docstring (:1-15): a THIN ORCHESTRATOR composing the SAME building blocks as the
backend.main CLI (validator registry -> segmenter -> report), never a forked pipeline. Parser at
build_parser() (:302).\n\nINFER ? cmd_infer (:88): `python -m backend.worker infer --video-uri
<uri> --out-uri <prefix> [--segmenter] [--config] [--scratch DIR] [--max-frames] [--set k=v]`.
Flow: (1) load_config + _apply_overrides (:49, dotted-path setattr onto the typed pydantic
```

config, _coerce bool/int/float at :36); set config.output.output_dir=scratch (:95). (2) PULL: storage.fetch(video_uri, scratch/in.mp4) ? identity for local://, download for s3/gcs (:98). (3) IDENTITY: compute_video_id = whole-file MD5 (:101), the labels<->video join key; pulled bytes must be byte-identical. (4) VALIDATE via the SAME validator_registry as the main CLI (:105), write video row to a scratch SQLite DB (:107). (5) SEGMENT: segmenter_registry.get(seg_name or config.indexing.default_segmenter).process_video(...) (:120-127). (6) emit_indexer_report in a finally (:136, parity with main CLI, never raises). (7) SERIALIZE outputs/<video_id>/[intervals.csv, report.json] ? intervals.csv is decimal-second [start,end)+shot_count+ending_reason, the join-friendly schema matching the rally-annotator labels (_write_intervals_csv :62). (8) PUSH each file via storage.put to <out_prefix>/outputs/<video_id>/<fn> (:149-154), idempotent on video_id.\n\nTRAIN ? cmd_train (:192): `python -m backend.worker train --corpus-uri <prefix> --out-uri <prefix> --version <semver> [--trajectory-source synth|detector] [--segmenter wasb\_hybrid] [--floor] [--fps] [--frame-width] [--config] [--scratch] [--set]`. Flow: list labels under <corpus>/labels/*.csv (storage.list_uris :217); REQUIRES >=2 for leave-one-video-out (:219). Two trajectory sources, train pipeline+harness identical either way (docstring :196-201): (a) synth (DEFAULT, the CPU \n"mock GPU\n") ? synth_trajectory_from_labels (stub_runner) makes deterministic label-consistent shuttle paths, NO GPU/WSL (:260-262); (b) detector ? REAL trajectories: resolve a DetectorRunner once (_resolve_train_detector :162), index videos/ by stem, fetch each video, probe real fps/width, runner.run_predict -> trajectory CSV (:242-258). Build manifest.json of specs (:269), then SHELL OUT (subprocess, because backend must NOT import training) to `python -m training.gen0.harness --manifest <m> --out <dir> --version <semver> [--floor <f>]` (:276-283). PUSH the versioned bundle to <out_prefix>/weights/<version>/<fn> (:289-294).\n\n_resolve_train_detector (:162) reuses the segmenter's _resolve_runner seam so worker and live CLI build the detector identically; rejects a non-trajectory segmenter (:179) and runs runner.healthcheck() before use (:184). Config knobs: --config (config.json), --set dotted overrides; the validated GPU-free/secret-free combo is RALLY_DETECTOR_IMPL=stub + --set indexing.skip_ai_handoff=true + storage.s3 settings (docstring :8-11, HANDOFF.md).",

"compute_abstraction": "Two ORTHOGONAL axes, deliberately separated (07 doc §2, lines 27-39): (A) ComputeBackend = WHERE a job runs (coarse, per-job: local | hosted | byo_worker / DO droplet / serverless) ? picks the machine; (B) DeviceContext = HOW a GPU stage runs ON that machine (fine, spec-aware, per-stage: probe the real GPU, resolve batch/precision/chunk knobs). Conflating them is the named design trap.\n\nSTATE TODAY: ComputeBackend is PLANNED ONLY ? `grep ComputeBackend backend/` returns nothing; it is named in PLATFORM_ARCHITECTURE.md (registry local|hosted|byo_worker) and inventoried as a FUTURE registry alongside StorageBackend in 07 §1 (:19) and 01 §3.4. NO ComputeBackend code, NO DeviceContext/DeviceSpec/DeviceProfile code ? 07 doc is a PROPOSAL (status :3) to be built at M3.5, NOT M0 (07 §6 :153, §5 caveat 2 :150).\n\nThe compute MAP / GPU-vs-CPU contract (07 §3 table lines 45-56, the load-bearing rule :57-59): \n"a GPU-bound stage is a pure, restartable unit whose only job is frames -> artifact (the Frame,Visibility,X,Y CSV or a feature JSON); everything downstream is device-agnostic CPU that reads that artifact.\n" GPU work is concentrated in the detection substrate ONLY ? person detection (torchvision, clean CPU fallback) + shuttle trajectory (WASB torch / TrackNet TF). Validators, ffmpeg chunk/stitch (libx264, nvenc optional), ByteTrack, windowing/candidate-merge, Gemini handoff (network I/O), DB/eval/calibration, Gen-0 head training (~4.5GB, minutes) are CPU/device-agnostic. This artifact boundary is EXACTLY where GPU and CPU work can live on different machines (the decoupling), and is already how EXPERIMENT_HARNESS/GOLDEN_DATA split today (07 §3 point 2).\n\nWhat abstracts compute placement TODAY (built): (1) the storage URI scheme ? local vs s3/gcs decides whether bytes are co-located or remote, transparently to the pipeline; (2) the detector_impl seam in trajectory_hybrid.py::_resolve_runner (:65): RALLY_DETECTOR_IMPL env (precedence) or indexing.detector_impl in {auto=WSL via _build_runner (:86), stub=CPU mock StubDetectorRunner (:77-81), native=Linux-native via _build_native_runner (:82-85)}. The native branch is the M3.5 hook but the base _build_native_runner REFUSES with NotImplementedError (\n"only WASB has a Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'\n", trajectory_hybrid.py:60-63) ? so LOCAL GPU = WSL today (auto), REMOTE/Linux GPU box = native (not yet built), CPU/CI = stub. The proposed DeviceContext layer (DeviceSpec/DeviceKnobs/DeviceProfile registry, 07 §4-5) would replace scattered device= logic and fix WASB's hardcoded device='cuda' with no CPU fallback (the one inconsistency, 07 §1 :16) ? but it only bites once the native runner lands, hence deferred to M3.5.",

"built_vs_next": "BUILT & MERGED to master (PRs #246-#254, suite 936 green per HANDOFF.md:9): the full bucket-driven train->infer pipeline running E2E on a CPU box with the GPU MOCKED (deterministic stub), validated on a DO droplet reading/writing DO Spaces:\n- Storage layer (#249): backend/storage/ ? local + S3 (AWS/DO Spaces/R2/MinIO) + GCS, URI-driven; gdrive is a stub.\n- Worker dispatcher (#251 infer, #254 train): backend/worker/__main__.py ? pull -> SAME pipeline -> push.\n- CPU-mock detector seam (#247): backend/pipeline/detectors/stub_runner.py via RALLY_DETECTOR_IMPL=stub /

indexing.detector_impl=stub.\n- Cross-platform canonicalization (#252); GPU-box provisioning scripts (#253): deploy/digitalocean/{bootstrap,mount_scratch,gpu_setup}.sh (NVMe scratch + native WASB conda env setup) ? scripts exist but the native runner that uses them does not.\n- Plan directory (#246): docs/DECOUPLED_COMPUTE/. \nVerified E2E (CPU, no GPU, no Gemini key, HANDOFF.md:26-30): worker train pulled 7 labels from Spaces -> synth_trajectory_from_labels (label-consistent CPU mock) -> shelled to training.gen0.harness -> LOVO over 7 videos (F1 1.000 on the mock, gate=no-ship by design) -> weights/ to Spaces; worker infer pulled a real video -> stub -> outputs/<md5>/ to Spaces.\n\nNEXT BUILD = M3.5, the ONE remaining piece for real (non-mock) training/inference (HANDOFF.md:50-65, 04 doc \$M3.5 :98): build NativeWasbRunner(DetectorRunner) in backend/pipeline/detectors/ ? sibling to WasbRunner but WITHOUT WslCommandMixin: run WASB in-process/native (no wsl / conda activate / to_wsl_mnt_path), device cuda/mps/cpu, weights via env (not ~/models), emit the IDENTICAL Frame,Visibility,X,Y CSV. Wire it into the existing native branch by overriding _build_native_runner in the WASB segmenter (today the base refuses, trajectory_hybrid.py:57-63). Acceptance gate = byte-parity: same clip -> WSL CSV (Windows) == native CSV (Linux) + identical candidate windows. Then wire a real-extract path into worker train/infer (--trajectory-source detector already exists; only the trajectory source flips, harness+push unchanged). Also proposed-not-built at M3.5: the DeviceContext/DeviceSpec/DeviceProfile device layer (07 doc) and the ComputeBackend registry. STILL NOTHING: no Dockerfile anywhere (README \$96 verification), no cost ledger / cost columns on the jobs table (05 doc), no remote inference endpoint / API-key-gated serverless, no bucket-triggered retrain flywheel. Gen-2 VLM fine-tune is explicitly OUT OF SCOPE for the \$200 budget.",

"single_box_fit": "There is NO literal \"single-box profile\" named in these docs ? the architecture is a control-plane / data-plane / compute-worker split (04 doc \$1 :12-27), and a SINGLE BOX is simply the DEGENERATE configuration of that same split where all three planes collapse onto one machine. The decoupling docs frame the END STATE (separate rented GPU + remote bucket); the single-box case is what already runs.\n\nHOW it maps, concretely:\n- STORAGE = LOCAL. Set storage.backend='local' (the DEFAULT, config/models.py:298). Then storage.fetch is an identity passthrough (no copy ? local.py:33) and storage.put no-ops when src==dst (local.py:42-47); parse_uri treats bare/Windows paths as local (ref.py:48-57). The worker's pull->pipeline->push collapses to \"read local file -> run -> write local dir\", byte-for-byte the pre-storage behaviour. FULLY BUILT, and the default.\n- WORKER IN-PROCESS / CO-LOCATED. `python -m backend.worker {infer|train}` runs the SAME registries as backend.main in the SAME process (worker docstring :12-15); the control plane (the shipped async jobs table + ThreadPoolExecutor on the owner's box, PLATFORM_ARCHITECTURE.md:60,475 ? single-tenant, local-only) and the compute worker are the same machine. The owner's CURRENT Windows/WSL dev box IS a single box: control plane + storage(local) + worker + GPU(WSL) all co-located. SUPPORTED today.\n- COMPUTE on a single box. The detector_impl seam picks the local detector: auto=WSL (the owner's GPU today) or stub=CPU. No ComputeBackend needed because there is only one machine ? ComputeBackend (WHERE) is the thing you ADD when the GPU moves off-box; on a single box it is identically the no-op \"local\" choice that need not exist yet (07 \$2, 01 \$3.4).\n- The DO CPU droplet 168.144.126.176 is ALREADY a working single-box instance of this in the cloud: BOTH /srv/khelsutra-site AND ~/rally (engine) live on that ONE box, and it ran the full train+infer E2E (GPU mocked + storage pointed at Spaces). Swap storage.s3->local and that same box is a pure single-box deployment.\n\nWHAT a single box ALREADY supports: the entire infer + Gen-0 train flow, end-to-end, mocked-GPU, on CPU; and on the owner's Windows+WSL box the REAL WSL detector via detector_impl=auto. WHAT IS MISSING for a single box to do REAL training/inference WITHOUT WSL (i.e. a single Linux GPU box): the M3.5 NativeWasbRunner ? until it lands, a non-Windows single box can only run stub (CPU mock) or must shell to WSL (Windows-only). Everything else a single box needs (storage=local default, worker, validators, Gen-0 harness, ffmpeg) is built. The cost-ledger / API-key / serverless / Dockerfile gaps are about MULTI-tenant remote serving, not the single-box path ? a single box does not need them.",

"file_refs": [
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\DECOUPLED_COMPUTE\\README.md",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\DECOUPLED_COMPUTE\\HANDOFF.md",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\DECOUPLED_COMPUTE\\01-EXISTING-PLANS-AND-GAP.md",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\DECOUPLED_COMPUTE\\02-COMPUTE-MAP-AND-CONTRACT.md",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\DECOUPLED_COMPUTE\\03-PLATFORM-OPTIONS.md",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\DECOUPLED_COMPUTE\\04-BUILD-AND-TEST-PLAN.md",

```

"C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\docs\\DECOUPLED_COMPUTE\\07-COMPUTE-ABSTRACTION.md",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\worker\\__main__.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\storage\\__init__.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\storage\\base.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\storage\\ref.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\storage\\s3.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\storage\\local.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\config\\models.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\pipeline\\segmenters\\trajectory_hybrid.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\pipeline\\detectors\\stub_runner.py"
]
},
"engineApi": {
  "process_flow": "FULL LOOP today (single FastAPI box, started via `python -m backend.main
--server` -> main.py:632-638 uvicorn on 127.0.0.1:8000). Step by step:\\n\\n1. UPLOAD (optional,
browser-side; chunked island in backend/api/uploads.py). A browser file is uploaded in
sequential parts <= security.max_part_mb (~95MB, edge-proxy cap): POST /api/upload/init
(uploads.py:64) validates extension (mp4/mov) + size, opens a `partial-<id>` temp under
security.upload_dir (output/uploads) and returns upload_id + next_part URL; PUT
/api/upload/{id}/part/{i} (uploads.py:89) appends raw bytes, parts STRICTLY sequential (409 on
gap), enforces per-part + whole-file caps (413 aborts); POST /api/upload/{id}/complete
(uploads.py:123) verifies part count, os.replace into upload_dir/<filename> (de-dupes name
collisions), computes video_id = MD5 (compute_video_id) and returns {path (absolute), video_id,
size_bytes, next:"POST /api/process with this path"}. DELETE /api/upload/{id} aborts + deletes
partial. Upload state is IN-PROCESS only (uploads.py:_uploads dict + lock) -> a restart drops
in-flight uploads (by design, single-box alpha). NOTE: the local /static UI does NOT use these
endpoints; it just takes a server-local path string.\\n\\n2. PROCESS submit (main.py:240 POST
/api/process, status_code=202). Fast-fail checks run synchronously in the request:
validate_input_path (+ optional security.allowed_video_root jail), os.path.exists -> 404,
unknown segmenter -> 400. Then it persists a job row in state 'queued' (db.create_job,
main.py:264) and fires `_get_executor().submit(_drain_due_jobs)`; returns 202 {job_id,
status:"queued", poll:"/api/jobs/{id}"}. ALL slow work ? including MD5 hashing of the file ?
happens on the worker thread (#16), not in the request.\\n\\n3. INDEX (worker thread,
_execute_processing main.py:125). Stages surfaced via progress callback (hashing -> validating
-> segmenting(<name>) -> finalizing): compute_video_id; db.get_video to detect a re-ingest;
registry.run_all_with_context runs the pluggable validators (resolution/fps/blur/scene-
cuts/court-relevance from config.validation). On any validation failure -> upsert video status
'failed', return failed result (no destruction of prior good segments). On pass -> upsert
'processed', resolve segmenter class from segmenter_registry, record pre-run watermarks
(db.segment_watermarks), run segmenter.process_video(video_id, path, player_pool, max_frames).
Destroy-AFTER-confirm: on non-empty result prune the prior rows, on empty/raise restore prior
rows (_finalize_reingest main.py:107). A per-video observability report is always emitted in
finally: (emit_indexer_report) and grafted as result["reports"]. Worker records terminal state
via db.mark_job_done (result carries the pipeline verdict in result["status"]) or
db.mark_job_failed.\\n\\n4. POLL (main.py:274 GET /api/jobs/{job_id}). Returns {job_id, status
(queued|running|waiting|done|failed = EXECUTION state), progress, video_id, error, result,
reports, created/started/finished_at}. UI loops every 3s until status done|failed; the
pipeline's own success/failed-validation verdict lives in result["status"], distinct from job
status.\\n\\n5. QUERY (main.py:331 POST /api/query) -> db.query_play_segments(video_id,
{server,receiver,duration_min,event_type}) -> {play_segments:[...]} (each with id, start_time,
end_time, server, receiver, events, metadata.shot_count).\\n\\n6. COMPILE reel (main.py:342 POST
/api/compile {video_id, segment_ids[], output_filename}). Looks up the video, intersects
requested segment_ids with stored segments, applies stitching.min_shot_count floor (unknown
shot_count exempt), builds (start,end) pairs, runs VideoCompiler(output_dir, begin/end_padding,
watermark) -> writes reel into output_dir; returns {status:"success", filepath}.
safe_output_filename rejects path traversal.\\n\\n7. DOWNLOAD reel (main.py:307 GET
/api/highlights/{video_id}/{filename}) -> streams the compiled file from output_dir via
FileResponse (video/mp4 or video/quicktime). The /static UI builds this URL after a successful
compile to offer a download link.\\n\\n(There is ALSO a fully-decoupled path the FastAPI does NOT
use: `python -m backend.worker {infer|train}` pulls inputs from the StorageBackend
(backend/storage/), runs the SAME pipeline, pushes results back ? that is the engine-decoupling
track, separate from this FastAPI loop.)",

```

```

"endpoints": [
  "GET / (main.py:52) -> serves backend/api/static/index.html (the SPA shell)",
  "GET /api/config (main.py:121) -> returns the live typed AppConfig (global_config) as
JSON",
  "POST /api/process (main.py:240, 202) -> req {video_path, player_pool?, max_frames?,
segmenter?}; fast-fail path/exists/segmenter checks, creates 'queued' job, kicks a drain; resp
{job_id, status:'queued', poll}",
  "GET /api/jobs/{job_id} (main.py:274) -> resp {job_id,
status(queued|running|waiting|done|failed), progress, video_id, error, result, reports,
created/started/finished_at}; 404 unknown",
  "POST /api/query (main.py:331) -> req {video_id, server?, receiver?, duration_min?,
event_type?}; resp {play_segments:[...]}",
  "POST /api/compile (main.py:342) -> req {video_id, segment_ids[], output_filename}; runs
VideoCompiler with stitching paddings+watermark; resp {status:'success', filepath}; 404 no video
/ 400 no matching segs or all below min_shot_count",
  "GET /api/highlights/{video_id}/{filename} (main.py:307) -> streams compiled reel from
output_dir (FileResponse); 404 if not compiled yet",
  "POST /api/upload/init (uploads.py:64) -> req {filename, total_size_bytes?}; validates
ext/size; resp {upload_id, max_part_mb, next_part}",
  "PUT /api/upload/{upload_id}/part/{index} (uploads.py:89) -> raw bytes body, strictly
sequential parts; resp {received_parts, received_bytes}; 409 out-of-order, 413 over-cap
(aborts)",
  "POST /api/upload/{upload_id}/complete (uploads.py:123) -> req {total_parts}; assembles
file; resp {path(absolute), video_id, size_bytes, next}",
  "DELETE /api/upload/{upload_id} (uploads.py:147) -> aborts in-flight upload + deletes
partial; resp {status:'aborted'}"
],
"job_worker": "In-process async worker, backend/api/job_worker.py (re-exported on
backend.main so monkeypatch seams work). ONE module-global ThreadPoolExecutor with max_workers=1
ON PURPOSE (job_worker.py:30-38): a single self-hosted box serializes GPU work and the Gemini
provider is rate-fragile under parallel uploads ? so jobs run STRICTLY ONE AT A TIME (segmenters
parallelize their own AI handoff internally via ai_max_workers). NO central scheduler: the DB
jobs table IS the clock. Submission (/api/process and server startup) just calls
_get_executor().submit(_drain_due_jobs). _drain_due_jobs (job_worker.py:100) is one tick: loop
db.claim_due_job() (atomic compare-and-set on (id,status) flipping queued|due-waiting ->
running, earliest-due first, so concurrent workers never double-claim) -> _run_claimed_job for
each until nothing runnable. _run_claimed_job (job_worker.py:72) reconstructs the ProcessRequest
from the row, runs _execute_processing with a progress cb that writes db.set_job_progress, then
db.mark_job_done(result) (terminal 'done' = worker finished; pipeline verdict is in
result['status']) or db.mark_job_failed on a raise. WAIT_AND_RESUME hook: if the pipeline raises
JobWaiting(delay_sec) (job_worker.py:41) the job is parked via db.set_job_waiting (status
'waiting' + next_poll_at + progress) and re-claimed when due ? but NO production segmenter
raises this today, so it is dormant. Self-scheduling: after each drain,
_schedule_next_drain_if_waiting (job_worker.py:124) computes db.seconds_until_next_due and
_arm_wake_timer arms ONE daemon threading.Timer (clamped to _MAX_DRAIN_WAKE_SEC=60s) to re-
submit a drain, so parked work resumes with no further client request. CRASH RECOVERY
(main.py:556 + db.fail_stale_jobs job_worker n/a -> database.py:613): at startup any job left in
'queued' OR 'running' from a dead process is swept to 'failed' (the executor was in-process, so
it's truly dead) ? pollers see a terminal state instead of a hang. CAVEAT: fail_stale_jobs
sweeps only queued/running, NOT 'waiting'; startup also fires one _drain_due_jobs to reclaim due
waiting jobs + re-arm the wake timer (main.py:637), but a 'waiting' job whose timer died and is
not yet due relies on that startup drain + the next submit to get re-armed. Single-process only:
all this state (executor, timer, uploads dict) is per-process ? there is no multi-box
coordination beyond the DB row-claim.",
"segmenter_selection": "SELECTION: per-request `segmenter` field (POST /api/process) wins;
else config default. config.json sets indexing.default_segmenter='gemini' (config.json:26)
[NOTE: the Pydantic built-in default in backend/config/models.py:122 is 'yolo_hybrid', but
config.json overrides it to 'gemini']. main.py:150 reads
getattr(global_config.indexing, 'default_segmenter', 'motion'). Registered names
(backend/pipeline/segmenters/__init__.py): motion, gemini, yolo_hybrid, tracknet_hybrid,
wasb_hybrid, fusion_hybrid (each self-registers via segmenter_registry.register). Unknown name
-> 400 at submit (main.py:256) or defensive 'failed' in-worker (main.py:172).\n\nWHAT EACH
NEEDS:\n- gemini (gemini.py): pure cloud. Requires GEMINI_API_KEY env (gemini.py:86-95; resolved
as <PROVIDER>_API_KEY). No GPU. Uploads re-encoded chunks to Google. This is the config.json
default.\n- motion: pure-CPU OpenCV motion heuristic; no GPU, no key.\n- yolo_hybrid /

```

tracknet_hybrid / wasb_hybrid / fusion_hybrid: two-stage trajectory hybrids
(TrajectoryHybridSegmenter, trajectory_hybrid.py). Stage 2 (shuttle/person trajectory ->
candidate windows) needs a DETECTOR RUNNER; Stage 3 is the AI handoff (Gemini) for semantic
boundaries, which needs GEMINI_API_KEY UNLESS indexing.skip_ai_handoff=true (config
models.py:196), in which case candidate windows ARE the rallies ? no provider, no key, nothing
uploaded (the pure `gemini` segmenter ignores skip_ai_handoff).\n\nDETECTOR RUNNER selection
(trajectory_hybrids only; trajectory_hybrid.py:65 _resolve_runner). Precedence:
RALLY_DETECTOR_IMPL env var > indexing.detector_impl (config default 'auto'):\n - 'auto'
(default) -> the real WSL runner via subclass _build_runner: wasb_hybrid -> WasbRunner (WSL2 +
conda env 'wasb' + pretrained WASB weights, GPU); tracknet -> TrackNet via WSL. NEEDS a GPU +
WSL2 + conda env (config indexing.wasb/tracknet blocks). This is the production path NOT
available on the CPU droplet.\n - 'native' / 'native_wasb' -> NativeWasbRunner
(native_wasb_runner.py) via _build_native_runner ? Linux-native, no WSL/conda-activate, GPU box
(M3.5, the NEXT build). ONLY wasb/fusion(substrate=wasb) support native (trajectory_hybrid.py:60
base refuses with a clear error; tracknet/yolo native -> NotImplementedError).\n - 'stub' ->
StubDetectorRunner (stub_runner.py): deterministic CPU MOCK, no GPU/WSL ? makes the whole hybrid
pipeline runnable/regression-testable on a CPU box/CI ("mock a GPU with a CPU"). This is what
the decoupled CPU-droplet e2e used (RALLY_DETECTOR_IMPL=stub + skip_ai_handoff to also drop
Gemini).\nSo the only GPU-free, key-free way to exercise a hybrid end-to-end is
detector_impl=stub + skip_ai_handoff=true. gemini needs the key (no GPU). motion is the only
fully-local, no-key, no-GPU REAL segmenter.",
"ui_drive_gaps": [
"EXISTS: a static SPA (backend/api/static/index.html + app.js + style.css) already drives
the CORE loop ? process by server-local path (POST /api/process + 3s poll of /api/jobs/{id}),
render validation results, render rally cards, filter via /api/query, select segments, POST
/api/compile, and build the /api/highlights download link. So index->reel is covered for an
already-on-server file.",
"GAP ? browser upload not wired: the chunked upload endpoints
(/api/upload/init|part|complete|abort) EXIST and work, but app.js never calls them; the UI only
takes a typed server-local path string. A real UI needs a file-picker that chunks <=max_part_mb,
drives init/part/complete, then feeds the returned absolute path into /api/process. (init
exposes max_part_mb to size the chunks.)",
"GAP ? no library/list endpoints: there is NO GET /api/videos and NO GET /api/jobs (list).
A UI can only re-find a video if it kept the video_id in memory; on reload everything is lost.
Need list-all-videos (with status) and list-all-jobs (history) endpoints + a backing db query ?
currently only get_job(id)/get_video(id) single-row lookups exist.",
"GAP ? segmenter/config not selectable in UI: /api/process accepts a `segmenter` field and
GET /api/config exposes the registry/default, but app.js hardcodes only {video_path,
player_pool} and never lets the user pick gemini vs wasb_hybrid vs motion, toggle
skip_ai_handoff, or see which detector_impl/GPU/key the box supports. A full UI needs to surface
available segmenters (from /api/config or list_available) + their requirements.",
"GAP ? query response field mismatch / no 'list all segments' refresh: app.js reads
data.rallies on /api/query (line 344) but the API returns {play_segments}; and the initial card
list comes only from the process result, so there is no way to re-load a previously indexed
video's segments without re-processing (no GET segments-by-video endpoint).",
"GAP ? progress is coarse + poll-only: /api/jobs returns a single free-text `progress`
stage string (hashing/validating/segmenting/finalizing); there is no percentage, no per-stage
ETA, no streaming (SSE/WebSocket). A real UI shows just a spinner with the stage text. Also no
cancel-job endpoint.",
"GAP ? no auth / multi-user / per-user scoping: FastAPI has NO auth and CORS
allow_origins='' (main.py:42-46); upload_dir + output_dir are global (per-user partitioning is
explicitly deferred to PR-3). A hosted UI can't isolate tenants today.",
"GAP ? reels not discoverable: /api/highlights/{video_id}/{filename} streams a reel only
if you already know the exact output_filename you compiled with; there is no list-compiled-
reels-for-video endpoint, so a UI can't show a gallery of previously made reels after reload.",
"GAP ? single-box concurrency ceiling: worker is max_workers=1 in-process, so the UI must
communicate 'one job at a time' (a 2nd submit queues behind the 1st); no progress/queue-position
is exposed, and a server restart fails in-flight jobs (UI should surface 'failed: interrupted by
server restart').",
"GAP ? decoupled storage/worker path is invisible to the UI: the StorageBackend + `python
-m backend.worker` path (S3/GCS/DO Spaces) is entirely separate from the FastAPI; the UI drives
only the local-filesystem FastAPI loop. Driving the decoupled GPU-worker loop from a UI would
need new orchestration/endpoints that don't exist yet.",
],
"file_refs": [

```

"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\main.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\api\\uploads.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\api\\job_worker.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\config.json",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\config\\models.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\pipeline\\segmenters\\base.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\pipeline\\segmenters\\__init__.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\pipeline\\segmenters\\trajectory_hybrid.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\pipeline\\segmenters\\wasb_hybrid.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\pipeline\\segmenters\\gemini.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\infrastructure\\database.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\api\\static\\app.js",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\api\\static\\index.html",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\pipeline\\detectors\\stub_runner.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\pipeline\\detectors\\native_wasb_runner.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\storage\\"
]

```

```

},
"hosting": {
  "exposure_auth": "INTENDED MODEL = single-tenant, private/invite-only, NO real auth shipped yet. The engine FastAPI has ZERO auth and a wide-open CORS: backend/main.py:42-43
`allow_origins=[\"*\"]`, `allow_credentials=False` (only one \"auth\" hit in main.py, in a comment at :39 ? no Depends/HTTPBearer/api_key guard exists). The dev SPA proxy also runs with no auth (khelsutra web/README.md:2 \"no auth locally (DEV_USER_EMAIL)\").\n\nThe DESIGNED (not-yet-built) exposure model is in docs/HOSTING_PLAN.md (status \"PROPOSAL ? awaiting owner ratification\"): Alpha = Cloudflare edge (free) ? DNS/TLS + **Cloudflare Access email-OTP allowlist** in front of BOTH app.khelsutra.guru (Pages SPA) and api.khelsutra.guru (HOSTING_PLAN.md:26-31); a **Cloudflare Tunnel** (`cloudflared`, outbound-only, \"No inbound ports opened ? Home IP never exposed\" :36) reaches FastAPI :8000 on the box. Auth = backend trusts the `Cf-Access-Authenticated-User-Email` header / verify the Access JWT for per-user scoping (HOSTING_PLAN.md:37-38). 100 MB edge body cap ? ?90 MB chunked upload (:39). Go-live checklist (:102-108) DEMANDS the auth/exposure hardening that does NOT yet exist: Access allowlist ON, CORS narrowed to https://app.khelsutra.guru, rate limit + per-user upload quota, `security.allowed_video_root`, rotate the Gemini key. Beta graduates auth to Firebase Auth + GCS direct upload (HOSTING_PLAN.md:46-67). PLATFORM_ARCHITECTURE.md:62 confirms \"API ? no auth\" is a known gap; auth is P1 (build-vs-buy Auth0/Clerk/Supabase, $8.3). Khelsutra marketing site is \"private/invite-only\" (KHELSUTRA_PRODUCT_OVERVIEW.md:41, :57). NET: a publicly-reachable box today is unauthenticated; the only intended gate is Cloudflare Access (edge), not anything in the app.",

```

```

  "locality_model": "docs/VIDEO_LOCALITY_MODEL.md (status \"THINKING DOC\", no greenlight). Central thesis ($1, :30): \"timestamps are the product; the video is dead weight\" ? the customer receives {rally intervals, report, reel} (kilobytes); only the highlight compiler needs the original full-res bytes, and that is pure ffmpeg that can run wherever the original lives. TODAY (L3, $0 :10-18): the chunked-upload flow assembles the full file under `security.upload_dir` (default output/uploads), registers by MD5, and **nothing ever deletes it** (retention = acknowledged gap, OQ1); the Gemini path also ships ?1280px re-encoded chunks to Google's Files API. Current cap = `security.max_upload_mb` default **4096 MB** ($0 :21), so a 10 GB upload is impossible today.\n\nThe \"locality ladder\" ($2 table :53-60): L3 full upload (built, capped, wrong for big files) ? **L2 proxy upload** (timeline-identical ?1080p re-encode, ~1-2.5 GB, the recommended alpha default $7 :140) ? L1 chunks-only / L1-direct (chunks straight to Gemini) ? **L0 fully local** (nothing leaves; the owned on-device model ? the desktop-app north star) ? L4 sneakernet (a hard drive; how the n=3 pilot runs today). Privacy framing: \"strictly no bytes leave only at L0\" and even L0 leaks ?1280px chunks to Google while Gemini is the brain ? must be in consent language (OQ3 :129). Storing the proxy not the original = ~10x less disk + smaller privacy surface ($3 :71-72), and the review UI prefers streaming a 1080p proxy (OQ6 :132). For a UI streaming source: L2 streams the proxy (good); L1 has only chunks ?

```

review degrades ? this argues L2 as the default playback/clip source. Reel delivery undecided (OQ7 :133): server-compiled proxy reel (shareable 1080p) vs intervals-back + client cuts full-res. Consent/marketing (KHELSUTRA_PRODUCT_OVERVIEW.md:101-103): \"handled securely on our infrastructure\", never published, minors' footage never used to improve the tool, training opt-in only.

"droplet_layout": "ONE shared CPU droplet 168.144.126.176 (DigitalOcean, Ubuntu 24.04, 2 vCPU / 3.8 GB, **NO GPU** ? SETUP_NEW_MACHINE.md:7) hosts BOTH:\n1) **The rally engine** at `~/rally` (git clone or scp'd archive; SETUP_NEW_MACHINE.md §3 :67-77). FastAPI on **port 8000** (HOSTING_PLAN.md:31; khelsutra web/README.md:2 dev proxy ? http://localhost:8000). Bootstrap = deploy/digitalocean/bootstrap.sh (mirrors CI: ffmpeg + libgl1 + .venv + CPU-or-CUDA torch + `pip install -e .[dev]`; verified torch 2.12.1+cpu, 922 passed/11 skipped). Storage layer = backend/storage/ (base/local/s3/gcs/gdrive/ref); worker = `python -m backend.worker {infer|train}` (backend/worker/`\_\_main\_\_`.py), URI-driven. Verified e2e 2026-06-20 reading/writing real DO Spaces (sgp1), GCS emulator, MinIO ? with the GPU MOCKED (`RALLY\_DETECTOR\_IMPL=stub`, `indexing.detector\_impl=stub`, `skip\_ai\_handoff=true`; SETUP_NEW_MACHINE.md §6a :120-143).\n2) **The khelsutra marketing/request-access site** at `~/srv/khelsutra-site` (isolated dir; PORTABILITY.md:32), a dependency-free Node `site/server.js` (node:http + node:sqlite) on **port 8787***, under systemd unit `khelsutra-site`, live at http://168.144.126.176:8787 (khelsutra/NEXT_STEPS.md:11-12; site/scripts/droplet-smoke.sh defaults REMOTE_DIR=~/srv/khelsutra-site PORT=8787). Same code also deploys to Cloudflare Workers+D1 (PORTABILITY.md two-hosts table); the VM is the \"if Cloudflare acts out\" fallback.\n\nGPU box = a SEPARATE, NOT-YET-BUILT machine (cattle): a DO **GPU Droplet**, RTX 4000 Ada 20 GB ~\$0.76/hr, in a GPU region (tor1/nyc2/atl1, NOT blr1 ? you cannot add a GPU to a CPU droplet; SETUP_NEW_MACHINE.md:43-47). Port procedure §9 :226-256: provision ? code to ~/rally ? `mount\_scratch.sh` (formats-mounts the 2nd raw NVMe at /scratch, ephemeral, NOT in fstab; `export TMPDIR=/scratch`) ? `bootstrap.sh --gpu` (CUDA 12.4 torch) ? `gpu\_setup.sh` (a SEPARATE miniconda env \"wasb\": py3.8 + torch 1.11.0+cu113, clones nttcom/WASB-SBDT, needs wasb_badminton_best.pth.tar) ? the \"M3.5\" native-WASB enabler. The real Linux-native DetectorRunner (NativeWasbRunner) is the NEXT build, not yet landed.\n\nSECRETS (the only non-reproducible items, never in repo): GEMINI_API_KEY at /root/.keys/GEMINI_API_KEY (chmod 600) and bucket creds in ~/.aws/credentials [default] (chmod 600) (SETUP_NEW_MACHINE.md:238-244). Bucket layout: `videos/ labels/ manifest.json weights/ outputs/<video\_id>/` (:210). Teardown = `doctl compute droplet delete` (cattle).\",

"desktop_vision": "YES ? a \"locally installed\" cross-platform desktop app IS already envisioned, in docs/DESKTOP_APP_PLAN.md (status \"DRAFT ? owner-gated; nothing built; the entire build is [design]\", :3-7). Pitch (§0 :11-12): \"Plug in your card, get your highlights\" ? package the mature local pipeline (WASB shuttle detector + windowing + ffmpeg stitch, already runnable offline via tools/generate_highlights.py --skip-ai-handoff) for non-technical recreational players; plug in USB/SD ? one click ? get back only the reels, 100% on-device, NO upload, NO copy of the 4K originals. It explicitly = the `local`/`LocalDesktop` ComputeBackend of PLATFORM_ARCHITECTURE.md §3.2 made into a product, and realizes the **L0 tier** of VIDEO_LOCALITY_MODEL (§4.3 :101-102, and DESKTOP_APP_PLAN header :6).\n\nSTACK (§3.4 :61-64, §4.4 :104-105): app shell = lean \"CLI + installer MVP wrapping the existing `indexer` console entry / generate_highlights.py ? graduate to a **Tauri** GUI (Rust + system webview, ~3-10 MB, MIT/Apache), with the **Python/FastAPI backend bundled as a localhost sidecar via PyInstaller** (PEP 621 pyproject + `indexer` entry point already shipped, #167). Electron is the fallback. Detector = a NEW **NativeWasbRunner** (DetectorRunner subclass, NO WSL ? drops the WSL2 dependency, D3 :37) lifting backend/pipeline/detectors/wasb_infer.py to native torch/onnxruntime per-OS, keeping the identical Frame,Visibility,X,Y CSV; windowing + VideoCompiler stitch reused verbatim. Licensing (D4 :38): MIT/Apache/BSD only, ffmpeg as an **LGPL build** (openh264/hardware encoders, no GPL libx264). \n\nCRUX/gate (§0 :14, §5, P0): compute on a typical no-GPU enthusiast laptop is the make-or-break unknown ? WASB is ~3.4x real-time on the owner's RTX 2070S [measured] but CPU/Apple-MPS throughput is UNVERIFIED; P0 is a feasibility spike that must measure it before any app code, and everything past P0 is gated. Relation to browser appliance: it's the privacy-max end of the SAME spectrum as the hosted SPA ? same pipeline, control/data-plane \"degenerates to one machine\" (§4.3 :102); Gemini stays an optional opt-in toggle, never required.\",

"single_tenant_assumptions": [

"NO app-layer auth exists: backend/main.py FastAPI ships open CORS allow_origins=['*'] and zero auth dependency ? a publicly-reachable box is wide open unless something external (the planned Cloudflare Access edge allowlist) gates it; HOSTING_PLAN go-live checklist (:102-108) lists CORS-narrowing + Access + rate-limit + per-user quota as REQUIRED-but-not-yet-done before exposure.\".

"The intended gate is edge-only and email-allowlist, not real multi-tenant identity: Cloudflare Access email-OTP for ?50 alpha testers, backend trusting Cf-Access-Authenticated-

User-Email (HOSTING_PLAN.md:37-38). Multi-tenant identity/RLS/per-tenant isolation is P1 and unbuilt (PLATFORM_ARCHITECTURE.md §4.2, §6).",

"Cloudflare Tunnel keeps the home/box IP unexposed and opens NO inbound ports (cloudflared dials out; HOSTING_PLAN.md:36) ? the single box is reachable only via the edge, never directly, in the intended design (the raw 168.144.126.176:8000/:8787 exposure today is a testbed, not the production posture).",

"Retention is an UNSOLVED guardrail gap: uploaded originals under output/uploads are never deleted (VIDEO_LOCALITY_MODEL §0 :10-18, Q01) ? a real privacy/liability hole for a single publicly-reachable box holding many users' video; default to keeping only proxies + intervals.",

"Minors are excluded by default from the training pool, and per-user/contributed-footage training requires a SEPARATE explicit opt-in (CLAUDE.md guardrails; PLATFORM_ARCHITECTURE.md §7 :516-526; KHELSUTRA_PRODUCT_OVERVIEW.md:98-103) ? never train on what merely passes through a tenant's bucket.",

"Paid LLMs (Gemini/OpenAI/Anthropic) are inference/serving/fallback ONLY, never a training source; even 'local' modes leak ?1280px chunks to Google's Files API on the Gemini path, which MUST be stated in consent language (VIDEO_LOCALITY_MODEL Q03; PLATFORM_ARCHITECTURE §5A teacher constraint).",

"Per-video cost ceiling needed so one box can't be surprise-billed: cap GPU/Gemini spend per video, ai_max_workers=1, serial uploads, bounded retry (VIDEO_LOCALITY_MODEL §4 :84-103).",

"Secrets live only on the box (GEMINI_API_KEY in /root/.keys chmod 600, bucket creds in ~/.aws/credentials chmod 600) ? never in repo/PR/config.json; rotate the previously-chat-shared Gemini key (SETUP_NEW_MACHINE.md:238-244; HOSTING_PLAN.md:106).",

"The droplet is a SHARED CPU-only testbed (MinIO 9000/9001, GCS emulator 9023, no GPU); real GPU WASB is a separate cattle GPU droplet in a non-blr1 region ? do not assume GPU or treat shared services as production."

```

],
"file_refs": [
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\HOSTING_PLAN.md",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\PLATFORM_ARCHITECTURE.md",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\VIDEO_LOCALITY_MODEL.md",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\DESKTOP_APP_PLAN.md",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\SETUP_NEW_MACHINE.md",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\KHELSUTRA_PRODUCT_OVERVIEW.md",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\deploy\\digitalocean\\README.md",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\deploy\\digitalocean\\bootstrap.sh",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\deploy\\digitalocean\\gpu_setup.sh",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\deploy\\digitalocean\\mount_scratch.sh",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\main.py",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\worker\\__main__.py",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\storage\\base.py",
  "C:\\Users\\avidu\\Projects\\khelsutra\\NEXT_STEPS.md",
  "C:\\Users\\avidu\\Projects\\khelsutra\\docs\\PORTABILITY.md",
  "C:\\Users\\avidu\\Projects\\khelsutra\\web\\README.md",
  "C:\\Users\\avidu\\Projects\\khelsutra\\site\\server.js",
  "C:\\Users\\avidu\\Projects\\khelsutra\\site\\scripts\\droplet-smoke.sh"
]
},
"uiRepo": {
  "site_ui_surface": "khelsutra repo (cwd C:\\Users\\avidu\\Projects\\khelsutra, private, all-rights-reserved, master) has THREE UI surfaces, two live + one scaffold:\\n\\n1) site/ ? the PUBLIC marketing landing page. SHIPPED + LIVE on khelsutra.guru. Runtime-portable static-asset + Worker app: `site/src/index.js` (Cloudflare Worker `fetch`) and `site/server.js` (node:http twin) both call the same `handleRequest(request, store)` over a `store` adapter (`site/src/store.js`: `d1Store` | `sqliteStore`) [PORTABILITY.md:8-23]. Public pages: `site/public/index.html` (landing) + `site/public/capture.html` (recording checklist) + `site/public/og.png`. It hosts the segmented Request-Access lead form ? D1 (`site/schema.sql`, `access_requests`) [REQUEST_ACCESS_FORM.md:42-66]. README.md:13: `site/ ? public marketing landing page (static; served at khelsutra.guru)`. Auto-deploys: merge to master ? Cloudflare Workers Builds (root site/) [NEXT_STEPS.md:9-12]. Also runs as a Node+SQLite twin live on the DO droplet at http://168.144.126.176:8787 (systemd khelsutra-site) ? the SAME droplet the engine

```

lives on.\n\n2) web/ ? the ALPHA React SPA. SCAFFOLD-ONLY today: web/ contains just README.md (no code yet). README.md:14: \"web/ ? React SPA (alpha; Vite + React + TS + Tailwind; Cloudflare Pages build root)\"; web/README.md:1-2: \"SPA scaffold lands here at execution-plan milestone M1 (Vite + React + TS + Tailwind). Dev: talks to the engine at http://localhost:8000 via Vite proxy; no auth locally (DEV_USER_EMAIL).\" This is where the operator/per-rally UI belongs.\n\n3) control-plane/ ? (beta) Cloud Run FastAPI control plane; README.md:16 ? not yet created.\n\nHow the appliance \"operator console\" relates to the public marketing site: they are deliberately SEPARATE surfaces with a gating boundary baked into the planned topology. The public marketing page (site/, khelsutra.guru, Cloudflare Pages/Worker, \$0) is UNAUTHENTICATED ? anyone can read it and submit the lead form. The operator console / appliance UI is a GATED APP: per HOSTING_PLAN.md:24-44 the alpha topology splits app.khelsutra.guru (Cloudflare Pages ? the web/ SPA, behind Cloudflare Access email-OTP allowlist) from api.khelsutra.guru (Cloudflare Tunnel ? FastAPI :8000 on the box). So the operator console is the web/ SPA sitting BEHIND Cloudflare Access, talking to the engine API ? NOT a public page. UI_PROPOSAL.md:33-37 confirms: SPA on static hosting, API on the box via tunnel, auth = Cloudflare Access email-allowlist (zero auth code; backend reads the authenticated-email header). Grounding caveat to record honestly: the engine's FastAPI today has NO auth (CORS '*'), so the Access/tunnel gate is the ONLY thing standing between the public internet and the appliance ? the console must assume the gate, not the app, is the tenancy boundary in alpha. The marketing site sells/funnels (Request-Access form captures GPU + storage answers = the BYO-compute/BYO-storage demand signal); the gated console operates the decoupled engine (the just-shipped storage + worker decoupling). A reasonable IA: keep the console as one or more authed screens in the SAME web/ SPA (Cloudflare Pages app root), distinct from site/ which stays the public Worker ? they already auto-deploy on separate roots (site/ Worker vs web/ Pages) so docs/web/ changes don't touch the live site/ Worker [PER_RALLY_SELECTION.md:108].\",

\"per_rally_relation\": \"The just-merged docs/PER_RALLY_SELECTION.md (PR #11, status IN PROGRESS, owner-approved 2026-06-20) is the FIRST screen of the broader operator console ? it is the \"pick rallies ? reel\" workhorse, not the whole console. Concretely:\n\n- PER_RALLY_SELECTION.md is the RallyPicker: a hybrid selectable rally grid + deterministic query bar that rides EXISTING engine endpoints with ZERO new engine code for its MVP ? POST /api/query ? select segment_ids ? POST /api/compile ? GET /api/highlights [PER_RALLY_SELECTION.md:12, \$4.1, D6]. It maps to alpha screen A5 Highlights (engine UI_PROPOSAL.md:47) and milestone M4 (UI_ALPHA_EXECUTION_PLAN.md), with its correction-loop extension realizing A4 Rally Review (UI_PROPOSAL.md:46).\n\n- An \"operator console\" for the DECOUPLED engine is a SUPERSET. PER_RALLY_SELECTION covers the curation-of-results step only; the console additionally needs the screens around it that UI_PROPOSAL.md:41-48 already enumerates ? A1 Matches (videos + verdict chips), A2 Upload & Process, A3 Job progress, A6 Report ? PLUS the genuinely new decoupled-era operations that PER_RALLY_SELECTION explicitly excludes: kicking off/monitoring worker infer/train jobs against a bucket, choosing a StorageBackend/bucket, per-video/per-run cost attribution, and weights/model versions. PER_RALLY_SELECTION is job-tethered by design (D10: carries result.video_id, no GET /api/videos) and deliberately punts multi-video home (P8, owner-gated), per-rally preview/timeline (P9), and the correction loop (P10) ? those punted milestones ARE the rest of the operator console.\n\n- So the relation is: per-rally selection = ONE screen (the reel-builder) of the console; the console wraps it with match list, upload, job-launch/monitor (the new decoupled worker verbs), report, and cost/model-version surfaces. The appliance tracked-doc should declare itself a peer-of PER_RALLY_SELECTION.md (and of REQUEST_ACCESS_FORM.md), reuse its verified engine-contract anchors (\$3, \$10) verbatim rather than re-deriving them, and adopt its honesty discipline (never surface motion-only server/receiver/events; never render a confidence meter from the Gemini string). It must NOT duplicate the RallyPicker design ? it should point to it as the embedded curation screen and own the job-orchestration/storage/cost surfaces PER_RALLY_SELECTION does not.\",

\"doc_template\": \"Follow the engine repo's docs/PROJECT_DOC_TEMPLATE.md (C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\PROJECT_DOC_TEMPLATE.md) ? the SAME template both existing khelsutra tracked docs mirror (PER_RALLY_SELECTION.md and REQUEST_ACCESS_FORM.md self-describe as \"mirrors the engine repo's PROJECT_DOC_TEMPLATE discipline\"). The exact structure to fill in:\n\nSTATUS HEADER (blockquote, 5 lines) ? > **Status:** `DRAFT` ? `<owner-gated? | in progress>` . **Owner:** Avi . **Created:** 2026-06-20 . **Last updated:** 2026-06-20 ; > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to docs/archives/ when DONE) ; > **Tracking anchors:** \$7 progress tracker is the source of truth; indexed in docs/README.md; pointer in NEXT_STEPS.md + memory current-state once work starts ; > **Relation to existing docs:** `peer-of PER\_RALLY\_SELECTION.md / REQUEST\_ACCESS\_FORM.md; extends UI\_PROPOSAL.md alpha screens>` ; > **Honesty note:** mark each claim [verified] / [researched] / [design]; not legal/financial advice.\n\nBODY SECTIONS (delete OPTIONAL ones that don't apply):\n\n- \$0 TL;DR (2?5 sentences: goal, approach, single most important caveat)\n\n- \$1 Problem & goal (why now; concrete user outcome; links to read to get

current))\n- §2 Decisions locked (table: # | Decision | Source / date | Implication) ? capture answered questions so they aren't relitigated\n- §3 Foundation ? research / prior art (OPTIONAL; include for engine-fact-heavy work, which this is ? use it for the verified engine/worker/storage contract, like PER_RALLY_SELECTION §3 did)\n- §4 Design / architecture (the plan + a REUSE MAP naming existing modules so new-vs-reused is explicit)\n- §5 Threat model / risk table (OPTIONAL ? relevant here: no engine auth, CORS '*', shared testbed droplet)\n- §6 Honest limits ? what this does NOT do (false-confidence guardrail)\n- §7 Deliverables & progress tracker ? SOURCE OF TRUTH (table: ID | Deliverable | Depends on | Gated? | Status | PR; legend ? Todo · ? In progress · ? Done · ? Blocked/gated; ONE small PR per row, branched from origin/master, NO stacking)\n- §8 Open questions ? owner / external (split blocking-gates from nice-to-knows; name who decides)\n- §9 Definition of done (explicit acceptance checklist; flip Status?DONE + archive when all ticked)\n- §10 References (Internal code anchors [verified] with paths; internal docs; external)\n- Changelog (dated entries at the bottom)\n\nHouse-voice rules from the template comment block (lines 9-16): §7 is the source of truth (one row per independently-shippable PR); keep it HONEST ? tag every claim [verified]/[researched]/[design], reflect real shipped state never aspirational; POINT to detail (code anchors), don't duplicate it; delete OPTIONAL sections that don't apply; move to docs/archives/ once Status=DONE (terminal-state only). After creating it, index it in khelsutra docs/README.md \"Tracked work (this repo)\" and add a pointer in khelsutra NEXT_STEPS.md (the same wiring PER_RALLY_SELECTION got).\",

\"reuse_opportunities\": [

\"Decoupled engine seams (just-shipped, master) ? the appliance's whole reason to exist: backend/storage/ StorageBackend ABC (base.py: fetch_to_local/put/exists/stat/list/delete + signed_url) with URI-driven local + s3 (AWS/DO Spaces/R2/MinIO) + gcs backends and lazy SDK import (__init__.py _backend_class), and the worker dispatcher backend/worker/__main__.py `python -m backend.worker {infer|train}` (pull from storage ? SAME pipeline building blocks ? push). The console drives THESE ? do not re-plumb storage or fork the pipeline; the worker is 'a thin orchestrator' over backend.main's blocks.\",

\"RallyPicker design + its verified engine-contract anchors ? reuse docs/PER_RALLY_SELECTION.md §3/§4/§10 (POST /api/query ? segment_ids ? POST /api/compile ? GET /api/highlights; GET /api/config min_shot_count; GET /api/jobs/{id} result.video_id) verbatim instead of re-deriving; embed RallyPicker as the console's curation screen rather than redesigning it.\",

\"Host-portability adapter pattern from PORTABILITY.md ? runtime-agnostic handleRequest(request, store) + d1Store/sqliteStore injection (site/src/store.js); the same 'logic-in-handler, host in adapter' shape is exactly how the StorageBackend registry already works in the engine, so the console can speak one URI contract across local/S3/GCS without host-specific branches.\",

\"Parity + CI discipline ? site/scripts/parity-check.sh + .github/workflows/parity.yml (scheduled live byte-for-byte drift check between the Cloudflare host and the droplet twin) and test.yml (npm test green on every PR, blocks merge); reuse this dual-host-parity + green-CI-gates-merge pattern for any console screen that has both a live droplet and a Cloudflare path. node --test runs BOTH host paths (PORTABILITY.md:25-30).\",

\"SPA scaffold + dev seam ? web/ (Vite + React + TS + Tailwind, Cloudflare Pages build root) with Vite proxy to engine :8000 and DEV_USER_EMAIL for no-auth local dev (web/README.md); the console screens land in this same scaffold (planned exec-plan M1) ? reuse the typed API client / useRallySelection() hook seams PER_RALLY_SELECTION P1-P2 introduce.\",

\"Tracked-doc + workflow conventions ? docs/PROJECT_DOC_TEMPLATE.md structure, the docs/README.md index + NEXT_STEPS.md pointer wiring, REQUEST_ACCESS_FORM.md's storage/GPU questions as the demand signal that justifies the appliance, and the one-PR-per-row-off-origin/master no-stacking discipline.\",

\"Gated-topology blueprint already written ? HOSTING_PLAN.md alpha topology (Cloudflare Access email-OTP in front of app.khelsutra.guru Pages + api.khelsutra.guru Tunnel ? FastAPI on the box, /api/health offline banner) is the ready-made gating model for the operator console; reuse it rather than inventing auth, and note the engine-has-no-auth caveat as a risk.\"

],

\"file_refs\": [

\"C:\\\\Users\\avidu\\\\Projects\\\\khelsutra\\\\README.md:11-16\",
\"C:\\\\Users\\avidu\\\\Projects\\\\khelsutra\\\\web\\\\README.md:1-2\",
\"C:\\\\Users\\avidu\\\\Projects\\\\khelsutra\\\\docs\\\\PER_RALLY_SELECTION.md:6,12,108,124\",
\"C:\\\\Users\\avidu\\\\Projects\\\\khelsutra\\\\docs\\\\PORTABILITY.md:8-30\",
\"C:\\\\Users\\avidu\\\\Projects\\\\khelsutra\\\\docs\\\\README.md:3-13\",
\"C:\\\\Users\\avidu\\\\Projects\\\\khelsutra\\\\docs\\\\REQUEST_ACCESS_FORM.md:5-9,42-66,88-94\",
\"C:\\\\Users\\avidu\\\\Projects\\\\khelsutra\\\\NEXT_STEPS.md:9-26,37-41\",
\"C:\\\\Users\\avidu\\\\Projects\\\\khelsutra\\\\.github\\\\workflows\\\\parity.yml\",

```

"C:\\Users\\avidu\\Projects\\khelsutra\\.github\\workflows\\test.yml",
"C:\\Users\\avidu\\Projects\\khelsutra\\site\\src\\store.js",
"C:\\Users\\avidu\\Projects\\khelsutra\\site\\src\\index.js",
"C:\\Users\\avidu\\Projects\\khelsutra\\site\\server.js",
"C:\\Users\\avidu\\Projects\\khelsutra\\site\\scripts\\parity-check.sh",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\PROJECT_DOC_TEMPLATE.md",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\UI_PROPOSAL.md:41-60",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\HOSTING_PLAN.md:22-44",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\DECOUPLED_COMPUTE\\README.md:18-29",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\DECOUPLED_COMPUTE\\HANDOFF.md:5-30,50-64",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\storage\\base.py:14-49",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\storage\\__init__.py:28-45",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\worker\\__main__.py:1-16"
]
}
}

```

THREE CANDIDATE APPROACHES:

```

[
{
  "name": "KhelSutra Box ? FastAPI-Native Single-Box Rally Indexer Appliance",
  "one_liner": "One FastAPI process (API + in-process worker) plus the existing same-origin static SPA, with StorageBackend=local, drives the entire upload -> index -> pick rallies -> reel -> download loop on a single box. gemini/stub make it run TODAY on the CPU droplet; native WASB swaps in on a GPU box with zero UI change.",
  "topology": "ONE python process: `python -m backend.main --server` -> uvicorn (today 127.0.0.1:8000, main.py:632-638). That single process IS the appliance: (a) the FastAPI API (process/jobs/query/compile/highlights/upload ? main.py + backend/api/uploads.py); (b) the in-process worker ? one module-global ThreadPoolExecutor(max_workers=1, job_worker.py:30-38) draining the SQLite jobs table via atomic claim_due_job; (c) the static SPA served SAME-ORIGIN ? FastAPI serves the shell at GET / (main.py:52) and mounts /static (main.py:50). There is NO separate static web server in the MVP ? the SPA (backend/api/static/) IS the operator console, served BY the engine, on the same origin. All state is on-box: SQLite (jobs/videos/segments via backend/infrastructure/database.py), output/uploads/ (assembled originals), output/ (reels), config.json. StorageBackend=local (the config default, models.py:298) so the worker's pull->pipeline->push degenerates to read-local/run/write-local (local.py:33 identity fetch, :42 no-op put). A reverse proxy (caddy/cloudflared) is OPTIONAL and appears ONLY when the box is exposed (see auth_exposure); the appliance itself stays one uvicorn process. This is the degenerate single-box collapse of the decoupled control/data/compute split ? all three planes on one machine ? which is exactly what already runs on droplet 168.144.126.176 (BOTH ~/rally engine and /srv/khelsutra-site live there today).",
  "request_path": "ONE origin, no second server in the MVP. The browser loads GET / (SPA shell, main.py:52) + /static/* (main.py:50) from uvicorn; every XHR (/api/process, /api/jobs/{id}, /api/query, /api/compile, /api/upload/*) hits the SAME uvicorn on the SAME origin ? so CORS is moot (the open allow_origins='' at main.py:42 becomes irrelevant when SPA and API share an origin; you do NOT rely on it). SOURCE video is never served to the browser in the MVP (privacy + no need): the reel preview/download is served by GET /api/highlights/{video_id}/{filename} which FileResponse-streams the compiled reel from output_dir (main.py:307-328) AFTER /api/compile writes it. This is the single most important simplification vs the gated-SPA-on-Pages topology in HOSTING_PLAN.md/khelsutra web/: by serving the SPA from FastAPI we get ONE origin, ONE deploy unit, ONE process to start ? no Vite proxy, no api.khelsutra.guru/app.khelsutra.guru split, no Cloudflare Pages build. The two-origin gated topology is the EXPOSURE upgrade, not the appliance core.",
  "compute_placement": "The SAME UI/process handles all three honestly because compute placement is chosen by config/env at the detector seam, never by the UI. (1) CPU droplet (TODAY, what ships): segmenter=gemini (needs GEMINI_API_KEY, no GPU, uploads <=1280px chunks to Google) for REAL semantic boundaries; OR any hybrid with indexing.detector_impl=stub + indexing.skip_ai_handoff=true (StubDetectorRunner, stub_runner.py ? deterministic CPU mock, candidate windows ARE the rallies) for a GPU-free, key-free, fully-local run that is HONESTLY a mock, not real shuttle tracking. motion is the only fully-local REAL segmenter (no key/GPU) but

```

low quality. (2) GPU box (native WASB): detector_impl='native' -> NativeWasbRunner (backend/pipeline/detectors/native_wasb_runner.py, the M3.5 piece on this very branch feat/native-wasb-runner) ? in-process torch cuda/mps/cpu, no WSL. Real shuttle trajectories; same /api/process path, same UI. (3) Owner's Windows+WSL box: detector_impl='auto' -> WasbRunner over WSL2+conda (real GPU today). The UI surfaces WHICH mode the box is in by reading GET /api/config (segmenter list + default + detector_impl) and labelling it ('Engine: gemini (cloud)' / 'wasb (native GPU)' / 'stub (CPU mock ? not real tracking)') so the operator is never misled. CRITICAL HONESTY: the CPU droplet CANNOT run real WASB at all (no GPU, native runner refuses without cuda; WSL is Windows-only) ? on the droplet the only REAL quality path is gemini (cloud). 'Local + private + real' requires the GPU box; the droplet gives you either real-but-cloud (gemini) or local-but-mock (stub).",

"auth_exposure": "The engine has ZERO app-layer auth and CORS '*' (main.py:42-43) ? a publicly-reachable box is wide open. So the appliance MVP MUST NOT be exposed naked. Three honest postures, pick per deployment: (A) BIND-LOCALHOST + TUNNEL (recommended for the shared droplet/alpha): keep uvicorn on 127.0.0.1:8000 (today's bind) and front it with a Cloudflare Tunnel (cloudflared, outbound-only, no inbound ports, home/box IP unexposed ? HOSTING_PLAN.md:36) + Cloudflare Access email-OTP allowlist on the hostname (HOSTING_PLAN.md:26-38). The gate is at the EDGE; the app trusts the Cf-Access-Authenticated-User-Email header. Zero auth code, reuses the written-but-unbuilt HOSTING_PLAN blueprint. Because the SPA is same-origin behind the SAME tunnel, ONE Access app covers both UI and API (simpler than the two-hostname plan). (B) EDGE AUTH via a tiny reverse proxy (caddy basic-auth / oauth2-proxy) when not on Cloudflare. (C) SINGLE-TENANT TOKEN: add ONE shared bearer token check as FastAPI middleware (smallest real in-app gate) for a truly private box. Whichever: the go-live checklist still applies (HOSTING_PLAN.md:102-108) ? narrow CORS if ever two-origin, set security.allowed_video_root jail (main.py:249), rate-limit, rotate the previously-chat-shared GEMINI_API_KEY (kept at /root/.keys chmod 600, never in repo). The boundary is the GATE (tunnel/proxy/token), NOT the app ? state this loudly. Single-tenant = no per-user isolation; uploads/outputs are global (deferred).",

"ui_console": "Augment the EXISTING same-origin SPA (backend/api/static/{index.html,app.js,style.css}) ? already drives process-by-path + 3s poll + rally cards + /api/query filter + select + /api/compile + download link) into a 5-screen operator console. Screens: (1) UPLOAD & PROCESS ? wire the file-picker to the EXISTING but un-called chunked endpoints (POST /api/upload/init -> PUT part/{i} sequential <=max_part_mb -> POST complete returns absolute path + video_id), then feed that path into POST /api/process; add a segmenter/skip_ai_handoff picker populated from GET /api/config so the operator sees+chooses the box's real capability. (2) PROCESSING/PROGRESS ? poll GET /api/jobs/{id} every 3s, show the free-text stage (hashing/validating/segmenting/finalizing), surface 'one job at a time' (max_workers=1) and 'failed: interrupted by restart' (fail_stale_jobs sweep). (3) RALLY INDEX + SELECTION ? REUSE the RallyPicker design from khelsutra docs/PER_RALLY_SELECTION.md VERBATIM (do NOT redesign): POST /api/query grid + deterministic filter bar -> select segment_ids -> /api/compile; honesty discipline (never show server/receiver/events for motion-only; no fake confidence meter from the Gemini string). (4) REELS/HISTORY ? the ONE genuinely-new bit needing a tiny engine addition: GET /api/videos + GET /api/jobs (list) + a list-reels endpoint so the console survives reload (today only single-row get_video/get_job exist ? ui_drive_gaps). MVP can defer history and key video_id in localStorage. (5) SETTINGS/COMPUTE+STORAGE ? read-only GET /api/config card showing engine mode (segmenter/detector_impl/has-GPU/has-key) + StorageBackend=local, so the operator knows whether results are real, mock, or cloud. Console lives in the engine's static dir for the MVP (one deploy); the khelsutra web/ React SPA is the post-MVP home (still same-origin-served or behind the tunnel).",

"reuse_vs_new": "REUSES (do not re-plumb): the entire FastAPI loop (main.py process/jobs/query/compile/highlights), the chunked-upload island (backend/api/uploads.py, endpoints already built+working, just un-called by the UI), the in-process worker (job_worker.py, max_workers=1, claim_due_job, crash-recovery fail_stale_jobs, wait/resume timer), StorageBackend=local default (no S3/worker-CLI needed for single-box ? worker pull/push degenerates to identity), the segmenter registry + detector_impl seam (gemini/stub today, native WASB later), VideoCompiler stitcher, the same-origin static-SPA serving (main.py:50/52), and the RallyPicker design (PER_RALLY_SELECTION.md §3/§4/§10 contract anchors verbatim). NET-NEW (small, additive): (1) UI: wire upload endpoints into app.js + segmenter/config picker + engine-mode banner; (2) engine: GET /api/videos, GET /api/jobs list, list-reels-by-video (3 read-only queries over existing DB tables) for reload-survival; (3) ops: a one-command launcher/installer (start uvicorn) + optional cloudflared/Access or a single-token middleware for exposure; (4) the tracked project doc (peer-of PER_RALLY_SELECTION.md per the template). NOT in scope, reused-by-pointing: the decoupled worker-CLI/S3 path (it's the multi-box future; single-box uses local), ComputeBackend/DeviceContext registries (07 doc, M3.5+, not needed when there's one machine), cost ledger / per-user isolation / Dockerfile (multi-tenant concerns, not single-box).",

```

"mvp_scope": [
  "Start it: one command `python -m backend.main --server` boots uvicorn = API + in-process
worker + same-origin SPA. StorageBackend=local (default). Verify GET / serves the console and
GET /api/config returns the engine mode.",
  "Engine: choose the box's compute mode by config/env only ? CPU droplet =>
segmenter=gemini (GEMINI_API_KEY set) for real boundaries, OR detector_impl=stub +
skip_ai_handoff=true for a GPU-free key-free MOCK run; GPU box => detector_impl=native
(NativeWasbRunner). No UI/code change between modes.",
  "UI screen 1 (Upload & Process): wire app.js to the EXISTING chunked endpoints (init ->
sequential parts <=max_part_mb -> complete -> absolute path) then POST /api/process with that
path; add a segmenter/skip_ai_handoff selector populated from GET /api/config; show an engine-
mode banner (cloud/native/mock).",
  "UI screen 2 (Progress): poll GET /api/jobs/{id} @3s, render stage text, state 'one job at
a time' + 'failed on restart'.",
  "UI screen 3 (Rally pick -> reel): embed the PER_RALLY_SELECTION RallyPicker verbatim ?
POST /api/query grid + filter bar -> select segment_ids -> POST /api/compile -> GET
/api/highlights/{video_id}/{filename} preview+download. Honesty rules from that doc.",
  "Engine add (small, for reload-survival): GET /api/videos + GET /api/jobs list + list-
reels-by-video over existing DB; (MVP may defer and use localStorage for video_id).",
  "Exposure: bind 127.0.0.1 + Cloudflare Tunnel + Access email-OTP allowlist (zero auth
code), OR a single shared-bearer-token FastAPI middleware for a private box. Never expose the
naked port.",
  "Tracked doc: create the appliance project doc as a peer-of PER_RALLY_SELECTION.md /
REQUEST_ACCESS_FORM.md (PROJECT_DOC_TEMPLATE.md), index in khelsutra docs/README.md +
NEXT_STEPS.md, one-PR-per-row off origin/master."
],
"pros": [
  "Smallest possible appliance: ONE process to start, ONE origin, ONE deploy unit ? no Vite
proxy, no Pages/Tunnel split, no second web server. The SPA is already served by FastAPI
(main.py:50/52).",
  "-90% reuse: the full CUJ loop (process/jobs/query/compile/highlights) AND the chunked-
upload endpoints already exist and are verified; the gap is mostly UI wiring + 3 read-only list
endpoints.",
  "Honest portability seam preserved: the SAME UI/process runs gemini (cloud), stub (mock),
or native WASB (GPU) chosen by config ? no fork. The GPU box is a config flip
(detector_impl=native), not a rewrite.",
  "Doesn't fight the decoupled model: single box = the degenerate StorageBackend=local + in-
process-worker configuration the decoupling docs explicitly describe as already-working; we ADD
nothing that conflicts with the S3/worker future.",
  "Runs TODAY on the existing CPU droplet (gemini for real, stub for mock) ? no GPU
procurement needed to demo the end-to-end CUJ.",
  "Exposure reuses the already-written HOSTING_PLAN blueprint (Tunnel + Access) with zero
auth code; same-origin makes it ONE Access app, simpler than the two-hostname plan."
],
"cons": [
  "On the CPU droplet you must pick your poison: real-but-cloud (gemini, leaks <=1280px
chunks to Google, needs key) OR local-but-mock (stub, NOT real shuttle tracking). 'Local +
private + real' is impossible there ? it needs the GPU box.",
  "max_workers=1: strictly one job at a time. A 2nd upload queues; the UI must communicate
this and there's no queue-position/cancel endpoint.",
  "No app auth: security rests entirely on the external gate (Tunnel/Access/proxy/token). A
naked exposed port is wide open (CORS '*'). The gate, not the app, is the tenancy boundary.",
  "Reload-survival needs the 3 new list endpoints; until then the console loses video_id on
refresh (localStorage is a stopgap).",
  "Retention gap: uploaded originals under output/uploads are never auto-deleted
(VIDEO_LOCALITY_MODEL Q01) ? a real disk/privacy liability on a shared box; needs a cleanup
policy.",
  "Single-process: a restart fails in-flight jobs and drops in-progress uploads (in-memory
_uploads dict). Acceptable for single-tenant alpha, not for unattended multi-user.",
  "Serving the SPA from FastAPI couples UI deploy to the engine repo (backend/api/static),
diverging from the khelsutra web/ React scaffold ? fine for MVP, a migration later."
],
"honest_caveats": [
  "The CPU droplet has NO GPU and cannot run real WASB: native runner needs cuda; WSL is
Windows-only. Real local shuttle tracking REQUIRES the separate GPU droplet (RTX 4000 Ada, non-

```

blr1 region, ~\$0.76/hr) ? not yet built; only the provisioning scripts exist.",

"NativeWasbRunner is the IN-PROGRESS M3.5 piece on this very branch (feat/native-wasb-runner, native_wasb_runner.py present). Its byte-parity acceptance gate (WSL CSV == native CSV) is not yet certified ? treat native mode as not-production-verified until that lands and is owner-checked on real video.",

"Stub mode is a deterministic MOCK that makes the pipeline runnable on CPU ? its rallies are NOT detections. The UI must label it as mock so no one mistakes it for real quality.",

"The engine has zero auth and CORS '*' (main.py:42-43). The Tunnel/Access exposure model in HOSTING_PLAN.md is a PROPOSAL ? not yet built. Do not expose the box without standing up the gate first; rotate the previously-shared Gemini key.",

"No Dockerfile exists anywhere ? 'one command' is `python -m backend.main --server` on a bootstrapped venv (deploy/digitalocean/bootstrap.sh), not a container. Containerizing is future work.",

"I verified the engine contracts by reading code (main.py same-origin SPA + 202/poll + compile + highlights; native_wasb_runner.py present; uploads endpoints exist). I did NOT run the server, exercise the chunked upload from a browser, or run real video through any segmenter ? those (and the GPU/native-parity path) need owner-side verification with the golden corpus.",

"No cost ceiling on the gemini path yet ? cap per-video Gemini spend (ai_max_workers=1, bounded retry) before any multi-user exposure so one box can't be surprise-billed.",

"Upload originals are never auto-purged; a single publicly-reachable box accumulating many users' video is a privacy/liability hole ? default to keeping only proxies+intervals and add retention before real use."

```
]
},
{
  "name": "KhelSutra Appliance - Reverse-Proxied Unified Origin",
  "one_liner": "A reproducible single-box appliance where Caddy serves the operator-console SPA and reverse-proxies /api/* to the no-auth engine FastAPI, with auth at the edge (Cloudflare Access or Caddy basicauth) so the engine never faces the public internet. Extends SETUP_NEW_MACHINE.md; drives the existing FastAPI loop today (gemini/stub on the CPU droplet) and the decoupled worker loop as native WASB lands, all from one origin.",
  "topology": "One box runs cooperating processes wired so ONLY the reverse proxy is public.\n\n(1) Caddy - the only process bound to public 0.0.0.0:443/80. Terminates TLS for app.khelsutra.guru (automatic ACME), serves the built SPA as static files, and reverse-proxies /api/*, /static/*, /api/highlights/* to the engine. Auth lives HERE.\n\n(2) Engine FastAPI: `python -m backend.main --server`, which hardcodes uvicorn host=127.0.0.1:8000 (main.py:638, VERIFIED). That localhost bind is the load-bearing safety property: the no-auth API (CORS '*', zero Depends guard, main.py:42) is reachable ONLY via Caddy's proxy, never directly on the public NIC. systemd unit khelsutra-engine, Restart=always; GEMINI_API_KEY from /root/.keys (chmod 600), bucket creds from ~/.aws/credentials.\n\n(3) In-process job worker - already inside the engine: the max_workers=1 ThreadPoolExecutor + DB-jobs-table-as-clock (job_worker.py). No separate worker daemon for the local FastAPI loop. The decoupled `python -m backend.worker {infer|train}` CLI is on-demand (subprocess), not a long-lived service in MVP.\n\n(4) Storage = LocalStorage: storage.backend='local' is the default (config/models.py:298); fetch/put are identity/no-op passthroughs (local.py:33,42) so the box filesystem (output/uploads, output_dir) IS the store. No S3 needed on a single box.\n\nThe marketing site (/srv/khelsutra-site node, :8787) stays a SEPARATE systemd unit on a SEPARATE subdomain (khelsutra.guru), untouched. The appliance owns app.khelsutra.guru only.",
  "request_path": "ONE origin to the browser; two upstreams behind it. Browser -> https://app.khelsutra.guru (Caddy, 443, public). Caddy routing: (a) / and SPA assets -> served from disk (the built web/ bundle, file_server, SPA fallback to index.html); (b) /api/* -> reverse_proxy 127.0.0.1:8000 (engine); (c) /api/highlights/{video_id}/{filename} -> same engine proxy, which streams the compiled reel via FileResponse (main.py:307). Because everything is one origin, CORS is a non-issue for the SPA (same-origin fetch) and the engine's permissive CORS '*' is moot from the browser's side - the real boundary is Caddy. Source/proxy video for the REVIEW player is NOT served by the engine today (no streaming endpoint exists); MVP plays reels only (the compiled output via /api/highlights). Inline review of a 1080p proxy is a deliberate post-MVP add (a new GET /api/proxy/{video_id} streaming endpoint + Caddy passthrough), tracked, not faked. So: one origin, one TLS cert, one auth gate; the SPA and the API look like one site to the user even though they are two processes.",
  "compute_placement": "Same SPA + same /api contract, three honest compute backends - the difference is ONLY which segmenter/detector the box can run, surfaced from GET /api/config (+ a new GET /api/capabilities) so the UI never offers a capability the box lacks.\n\n(A) CPU droplet (the live 168.144.126.176): NO GPU, NO local WASB. Honest options are (i) segmenter='gemini' (cloud brain, needs GEMINI_API_KEY, no GPU) for REAL rallies, or (ii) any hybrid with
```

detector_impl=stub + skip_ai_handoff=true for a deterministic CPU MOCK (regression/demo only, NOT real detection). The UI shows a 'CPU box - real detection via Gemini; local WASB unavailable' badge from /api/capabilities. This is the MVP target and it fully drives the CUJ.\n\n(B) GPU box (single Linux GPU droplet): segmenter='wasb_hybrid' with detector_impl='native' (the NativeWasbRunner now landing on branch feat/native-wasb-runner) runs real on-device shuttle detection, no WSL, no cloud. Same SPA, same /api/process; only /api/capabilities advertises wasb_hybrid as available. The owner's Windows+WSL box is the same story via detector_impl='auto' (WSL).\n\n(C) Remote GPU worker via bucket: the decoupled path. The console box stays CPU; it flips storage.backend to s3 (DO Spaces), and a SEPARATE GPU box runs `python -m backend.worker infer --video-uri s3://... --out-uri s3://...` pulling/pushing the bucket. The console then reads outputs/<video_id>/ back from the bucket. OUT of MVP (needs job-orchestration endpoints the engine does not have) but the design RESERVES the seam: the console's 'compute target' selector maps 1:1 to ComputeBackend (WHERE), which on a single box is the no-op 'local' choice (07 doc), so adding (C) later does not refactor the UI.\n\nThe ONE-UI-three-backends honesty rule: the console renders only what /api/capabilities exposes (available segmenters, GPU/native runner present, Gemini key set); it must NEVER render a wasb_hybrid option on a CPU box or imply real detection when detector_impl=stub.",

"auth_exposure": "Engine is no-auth BY DESIGN and stays that way; the gate is external. Two ratified-elsewhere options, pick per deployment:\n\n(1) Caddy edge auth (simplest, self-contained appliance): Caddy `basicauth` (bcrypt-hashed operator password in the Caddyfile) or `forward\_auth` to an OIDC proxy. Single-tenant, invite-only, zero engine code. Good for the owner-operated alpha.\n\n(2) Cloudflare Access (the HOSTING_PLAN.md alpha model): cloudflared Tunnel dials OUT from the box to Cloudflare (NO inbound ports opened, box IP never exposed, HOSTING_PLAN.md:36); Cloudflare Access enforces an email-OTP allowlist in front of app.khelsutra.guru; the box trusts the Cf-Access-Authenticated-User-Email header / verifies the Access JWT. With Tunnel, Caddy can even bind 127.0.0.1 only and cloudflared is the sole ingress.\n\nThe NON-NEGOTIABLE invariant either way: the engine binds 127.0.0.1:8000 (already true, main.py:638) and the DO/cloud firewall BLOCKS inbound 8000 (and the 9000/9001/9023 MinIO/GCS-emulator test ports) so the raw API is unreachable from the internet. The gate is the proxy/tunnel, NOT the app. Go-live checklist (reuse HOSTING_PLAN.md:102-108): Access/basicauth ON, firewall closes 8000, rotate the previously-chat-shared GEMINI_API_KEY, set security.allowed_video_root to jail /api/process paths. Single-tenant static bearer is the fallback if neither edge option is wanted (token in the SPA + a thin Caddy header check) - but edge auth is preferred because it keeps the engine truly auth-free.",

"ui_console": "The operator console = authed screens in the EXISTING khelsutra web/ SPA (Vite+React+TS+Tailwind, Cloudflare Pages root), behind the edge gate, talking to /api on the same origin. It is a SUPERSET of PER_RALLY_SELECTION.md's RallyPicker (which it EMBEDS, not reduplicates). Screens:\n\nn1. Upload & Process (A2) - file-picker that chunks <= max_part_mb via the EXISTING but un-wired /api/upload/init|part|complete (uploads.py), then feeds the returned absolute path into POST /api/process with a segmenter chosen from /api/capabilities. Wiring this is the single biggest net-new UI gap (app.js never calls upload today).\n\nn2. Processing / Progress (A3) - polls GET /api/jobs/{id} every 3s, shows the free-text stage (hashing/validating/segmenting/finalizing) + a 'one job at a time' note (max_workers=1) + an honest 'failed: interrupted by server restart' state (fail_stale_jobs sweeps queued/running on boot).\n\nn3. Rally index + selection - EMBED RallyPicker verbatim: POST /api/query -> select segment_ids -> POST /api/compile -> GET /api/highlights. Reuse PER_RALLY_SELECTION.md \$3/\$4/\$10 anchors; honor min_shot_count from /api/config; NEVER surface motion-only server/receiver/events or a fake confidence meter.\n\nn4. Reels / History - needs two SMALL net-new engine endpoints (GET /api/videos, GET /api/reels/{video_id}) so the console survives reload (today only get_job(id)/get_video(id) single-row lookups exist). Without them the library is amnesiac.\n\nn5. Settings (compute + storage) - read-only in MVP: render /api/capabilities (segmenters available, GPU/native present, Gemini key set, storage backend). The storage/compute-target SELECTORS (local vs bucket vs remote worker) are the reserved decoupled-era seam, shown DISABLED with 'single-box: local' until worker-orchestration endpoints exist.",

"reuse_vs_new": "REUSES (do not re-plumb): the entire FastAPI loop /api/process -> /api/jobs -> /api/query -> /api/compile -> /api/highlights (built, main.py); the in-process max_workers=1 worker + DB-as-clock (job_worker.py); the chunked upload endpoints (uploads.py - built, just un-wired in app.js); StorageBackend ABC + LocalStorage default + the `python -m backend.worker` dispatcher (the decoupled seam for later remote GPU); the detector_impl seam (auto/native/stub) incl. the NativeWasbRunner now landing on this branch; the web/ SPA scaffold + RallyPicker design (PER_RALLY_SELECTION.md, embedded); the host-portability adapter pattern (handleRequest+store) and parity/test CI gates from the khelsutra repo; the gated topology from HOSTING_PLAN.md; SETUP_NEW_MACHINE.md / deploy/digitalocean/*.sh as the runbook base.\n\nNET-NEW (small, additive): a Caddyfile + a Caddy systemd unit + a khelsutra-engine systemd unit (NO Dockerfile exists anywhere today - README §96; this design picks Caddy+systemd as the lighter

reproducible path over a compose file); GET /api/capabilities (derive available segmenters + GPU/native + key-present + storage backend from existing config/registry - thin); GET /api/videos and GET /api/reels/{video_id} (two small DB list queries - the only new engine DATA endpoints MVP needs); the web/ console screens (Upload wiring, Progress, Reels/History, Settings) around the embedded RallyPicker; a tracked project doc (peer of PER_RALLY_SELECTION.md, per PROJECT_DOC_TEMPLATE.md). Everything else is configuration, not code.",

"mvp_scope": [

"Caddyfile + two systemd units (khelsutra-engine, caddy) extending SETUP_NEW_MACHINE.md: one origin app.khelsutra.guru, TLS via ACME, /api/* + /static/* + /api/highlights/* -> 127.0.0.1:8000; engine stays bound to 127.0.0.1:8000 (already true); DO firewall blocks inbound 8000.",

"Edge auth ON: Caddy basicauth (single operator) for the self-contained appliance, OR cloudflared Tunnel + Cloudflare Access email-allowlist per HOSTING_PLAN.md. Rotate the GEMINI_API_KEY before go-live.",

"Wire browser upload in the web/ SPA: file-picker -> chunk <= max_part_mb via /api/upload/init|part|complete -> feed absolute path to POST /api/process (segmenter from /api/capabilities).",

"GET /api/capabilities (thin): available segmenters, GPU/native-runner present?, Gemini key set?, storage backend - so the UI only offers what the box can do (CPU droplet => gemini real or stub mock; never wasb_hybrid).",

"Drive the full CUJ on the CPU droplet end to end: upload -> POST /api/process segmenter='gemini' (real rallies, needs key) -> poll /api/jobs -> RallyPicker (/api/query, select, /api/compile) -> download reel (/api/highlights). Stub+skip_ai_handoff is the no-key demo fallback.",

"GET /api/videos + GET /api/reels/{video_id} (two small DB list endpoints) + a Reels/History screen so the console survives reload (today it is amnesiac on refresh).",

"Embed RallyPicker (PER_RALLY_SELECTION.md) verbatim as the selection screen; reuse its verified engine-contract anchors; honor min_shot_count; no fake confidence/empty motion fields.",

"A tracked project doc (PROJECT_DOC_TEMPLATE.md, peer of PER_RALLY_SELECTION.md) with \$7 progress tracker, one small PR per row off origin/master, indexed in khelsutra docs/README.md + NEXT_STEPS.md."

],

"pros": [

"Exposure-safe by construction: engine already binds 127.0.0.1 (main.py:638), so the no-auth API is unreachable except via the proxy; auth at the edge means ZERO auth code in the engine and no risk of a half-built in-app auth being the only guard.",

"One origin = no CORS to design, one TLS cert, one URL for the operator; the engine's permissive CORS '*' becomes irrelevant from the browser side.",

"Maximally reuses shipped code: the whole /api loop, the in-process worker, the upload endpoints, the storage+worker decoupling, the RallyPicker, and the SETUP_NEW_MACHINE runbook all stand - MVP is mostly config + 3 thin endpoints + UI wiring.",

"Honest about compute: the same SPA serves CPU droplet (gemini/stub), GPU box (native WASB), and future remote-worker without UI forks, because it renders only /api/capabilities.",

"Reproducible ops: Caddy+systemd extending the existing runbook is lighter than the (nonexistent) Dockerfile path and matches how the box already runs the engine + marketing site as systemd units.",

"Does not fight the decoupled architecture: single box is the degenerate (storage=local, compute=local) configuration of the same control/data/worker split; the storage-URI + ComputeBackend seams are reserved, not contradicted."

],

"cons": [

"Two net-new engine endpoints (GET /api/videos, GET /api/reels/{video_id}) are genuinely required for a non-amnesiac UI - the 'zero new engine code' claim that holds for RallyPicker does NOT hold for the console as a whole.",

"max_workers=1 ceiling: a 2nd concurrent operator/upload queues behind the 1st with no queue-position surfaced; fine for single-tenant, visibly slow if shared.",

"No Dockerfile means reproducibility leans on systemd+Caddyfile+bootstrap.sh being kept in sync by hand (a runbook, not an image); drift risk across boxes.",

"Inline review is reels-only in MVP - there is no proxy/source streaming endpoint, so the operator cannot scrub the original before compiling; adding it is a real (tracked) follow-up, not a config flip.",

"Edge auth is the ENTIRE tenancy boundary: if Caddy/Access is misconfigured or the firewall does not block 8000, the open engine is exposed - the go-live checklist must be

enforced, not assumed.",

"Progress is coarse (free-text stage, poll-only, no %, no cancel) - the UX shows a stage spinner, not a real progress bar, until SSE/cancel are added."

],

"honest_caveats": [

"The live droplet is CPU-only with NO GPU and NO local WASB. On that box, REAL rally detection requires Gemini (cloud, needs a key, ships <=1280px chunks to Google's Files API - must be in consent language); detector_impl=stub is a deterministic MOCK for demo/regression, NOT real detection. Do not let the UI imply otherwise.",

"The engine has ZERO app-layer auth and CORS '*' (main.py:42). Safety depends ENTIRELY on the engine staying bound to 127.0.0.1 + a firewall blocking 8000 + the edge gate. If any of those three slips, the box is wide open.",

"No Dockerfile exists anywhere (verified). The 'appliance' is a runbook (SETUP_NEW_MACHINE.md + Caddyfile + systemd units + bootstrap.sh), not a packaged image - reproducibility is procedural.",

"Uploaded originals under output/uploads are NEVER deleted today (VIDEO_LOCALITY_MODEL OQ1) - a real privacy/disk hole on a publicly reachable box. MVP must add at least a retention sweep or keep-proxy-only policy before exposing to non-owner users.",

"GET /api/videos, GET /api/reels, GET /api/capabilities, browser-upload wiring, and a retention sweep are NET-NEW and unbuilt - the console is not a pure UI-over-existing-API project end to end.",

"Native WASB (detector_impl=native) is landing on branch feat/native-wasb-runner but its acceptance gate is byte-parity on a REAL GPU box, verified by the owner - it is NOT exercisable on the CPU droplet, so the MVP single-box CUJ is proven only via gemini (real) or stub (mock).",

"The remote-GPU-worker compute path (C) needs job-orchestration endpoints that DO NOT exist (the FastAPI loop and the backend.worker bucket loop are separate today); the console's storage/compute selectors are a reserved seam shown disabled, not a working MVP feature.",

"Per-video Gemini/GPU cost has no ceiling or ledger today (no cost columns on the jobs table); a single box can be surprise-billed - cap ai_max_workers=1 (already), serial uploads (already), and add a per-video spend cap before wider exposure."

]

},

{

"name": "KhelSutra Studio - single-box rally indexer + reel console (decoupled-profile appliance)",

"one_liner": "One web console (the khelsutra web/ SPA, served behind Cloudflare Access) drives the existing FastAPI engine through its full upload->index->pick-rallies->reel->download loop on ONE box, where compute placement is a settings choice: the SAME UI watches job/output state whether rallies come from gemini/stub on the CPU droplet, native-WASB on a co-located GPU, or a remote GPU worker draining the shared bucket.",

"topology": "ONE box runs four cooperating things, mirroring the droplet today (168.144.126.176 already co-hosts /srv/khelsutra-site + ~/rally). (1) The khelsutra web/ SPA (Vite+React+TS+Tailwind, Cloudflare Pages build root) is the operator console - static assets, zero engine code. (2) The engine FastAPI 'python -m backend.main --server' on 127.0.0.1:8000 is BOTH the control plane (the shipped jobs table + ThreadPoolExecutor(max_workers=1) self-scheduling drain in job_worker.py; the DB jobs table IS the clock, no central scheduler) AND the data plane (uploads.py chunked upload, /api/compile, /api/highlights FileResponse stream). (3) Storage = the StorageBackend ABC; on a single box it is the DEFAULT storage.backend='local' (config/models.py:298), where fetch_to_local is an identity passthrough (local.py:33) and put no-ops on src==dst (local.py:42-47), so a worker pull->pipeline->push collapses to read-local -> run -> write output_dir, byte-for-byte the pre-storage behaviour. (4) Compute substrate = the detector_impl seam in trajectory_hybrid.py::_resolve_runner: gemini (cloud, no GPU), stub (CPU mock), auto=WSL, native=GPU. HONESTY: today the UI loop runs ENTIRELY through the in-process FastAPI worker - the jobs table stores a local video_path, NOT a storage URI - and the decoupled 'python -m backend.worker {infer|train}' is a SEPARATE one-shot CLI the FastAPI does NOT call. So the single-box MVP is the FastAPI loop with storage=local + segmenter=gemini-or-stub; the bucket worker is the same box SECOND mode for the BYO-GPU story, reached by config, not by a new always-on daemon (the UI<->bucket-worker bridge is net-new, deferred past MVP).",

"request_path": "ONE origin in the MVP, TWO in the gated topology. MVP (single box): the web/ SPA is served as static assets; all data calls hit the same engine FastAPI on :8000 (Vite proxy in dev, the HOSTING_PLAN tunnel in prod). A browser request reaches the engine as: SPA -> POST /api/process {video_path} -> 202 {job_id} -> poll GET /api/jobs/{id} every 3s until done|failed -> POST /api/query -> POST /api/compile -> GET /api/highlights/{video_id}/{filename} (FileResponse). SOURCE/REEL VIDEO is served by the engine box itself: the reel streams from /api/highlights; for review playback there is NO endpoint today (gap - see ui_console). The

GATED prod topology (HOSTING_PLAN.md:24-44) splits app.khelsutra.guru (Cloudflare Pages -> web/SPA) from api.khelsutra.guru (Cloudflare Tunnel -> FastAPI :8000), both behind Cloudflare Access email-OTP; that is two hostnames but still ONE engine box. Raw 168.144.126.176:8000 today has CORS '*' and zero auth - the tunnel+Access edge is the ONLY tenancy boundary, never the app.",

"compute_placement": "THREE placements, ONE UI, expressed as engine config the console surfaces from GET /api/config (it already returns the live typed AppConfig) - NOT three code paths in the UI. (A) CPU droplet (gemini or stub): segmenter='gemini' needs GEMINI_API_KEY, no GPU, uploads <=1280px chunks to Google (state in consent copy); OR a trajectory hybrid with RALLY_DETECTOR_IMPL=stub + indexing.skip_ai_handoff=true = the GPU-free, key-free CPU mock (the validated droplet e2e). Runs through the in-process FastAPI worker - SHIPPED, this is the MVP. (B) Co-located GPU box (native WASB): segmenter='wasb_hybrid' + indexing.detector_impl='native' -> NativeWasbRunner. BLOCKED: the base _build_native_runner REFUSES with NotImplementedError (trajectory_hybrid.py:57-63); native_wasb_runner.py is the M3.5 NEXT build, not landed. On Windows+WSL, detector_impl='auto' gives the REAL WSL detector today. Still runs through the in-process FastAPI worker once the runner exists. (C) Remote GPU worker via bucket (BYO-compute): storage.backend='s3'/'gcs' + 'python -m backend.worker infer --video-uri s3://.../videos/<md5>.mp4 --out-uri s3://...' on a separate GPU droplet; the UI watches outputs/<video_id> in the bucket instead of polling /api/jobs. The SAME web/ SPA works because the per-rally screen needs only {intervals, report, reel} - but the orchestration that enqueues a bucket job and reflects its state into the UI is NET-NEW (no /api/jobs over storage exists). MVP picks (A); (B)/(C) are the same console with compute as a setting, gated on M3.5 + the bridge.",

"auth_exposure": "Edge auth, NOT app auth - and say so loudly. The engine FastAPI ships allow_origins=['*'], allow_credentials=False, and ZERO auth dependency (main.py:42-46); a publicly-reachable :8000 is wide open. The single intended gate is Cloudflare Access email-OTP allowlist (<=50 alpha testers) in front of BOTH app.khelsutra.guru and api.khelsutra.guru, reached via a Cloudflare Tunnel (cloudflared, outbound-only, no inbound ports, home/box IP never exposed - HOSTING_PLAN.md:36). The backend trusts the Cf-Access-Authenticated-User-Email header for per-user scoping (alpha-grade tenancy - trustworthy ONLY because the tunnel accepts only Access-authenticated traffic). For the BYO-compute single-tenant case the recommended posture is: bind the engine to localhost + reach it via the Access-gated tunnel (no token in the app), and supply a single shared per-tenant token ONLY if a non-Cloudflare reverse proxy is used. Go-live checklist that does NOT exist yet and MUST precede exposure: Access allowlist ON, CORS narrowed to https://app.khelsutra.guru, basic rate limit + per-user upload quota, security.allowed_video_root jail set, GEMINI_API_KEY rotated (it was chat-shared). Treat the gate, not the app, as the boundary until app-layer auth (P1) lands.",

"ui_console": "Six screens in the web/ SPA; the MVP is the four that ride EXISTING endpoints with zero new engine code, embedding the already-approved RallyPicker as the curation screen. (S1) Upload: a file-picker that chunks <=max_part_mb via /api/upload/init|part|complete (these endpoints EXIST but app.js never calls them - the net-new client wiring), then feeds the returned absolute path into POST /api/process. Inline recording-guidelines checklist. (S2) Process/Progress: shows the coarse free-text stage from GET /api/jobs/{id} (hashing->validating->segmenting->finalizing) on a 3s poll + a 'one job at a time' notice (max_workers=1) and an honest 'failed: interrupted by server restart' state (fail_stale_jobs sweeps queued/running at startup). No %ETA/SSE today. (S3) Rally index + selection = EMBED docs/PER_RALLY_SELECTION.md RallyPicker verbatim: POST /api/query -> select segment_ids -> deterministic query bar (server/receiver/duration/event), honouring GET /api/config min_shot_count; NEVER render motion-only server/receiver/events or a confidence meter from the Gemini string. (S4) Reels/History: name reel -> POST /api/compile -> GET /api/highlights download link. (S5) Settings = compute+storage: read GET /api/config to SHOW the segmenter registry + default + detector_impl + whether a GPU/key is present, and let the user pick gemini vs wasb_hybrid vs stub + toggle skip_ai_handoff; storage backend shown read-only in MVP. (S6) Library/Matches: list all videos+verdict chips and a reels gallery. GAPS the console must own or stub honestly: NO GET /api/videos, NO GET /api/jobs list, NO list-reels, NO review-playback stream, NO cancel-job - S6 + reel gallery + review playback need net-new engine endpoints (declare them as gated deliverables, do not fake them).",

"reuse_vs_new": "REUSES (do not re-plumb): the StorageBackend ABC + URI scheme (backend/storage/, local default = identity) and the worker dispatcher backend/worker/__main__.py (pull->SAME pipeline blocks->push, a thin orchestrator never a fork); the entire FastAPI loop (process/jobs/query/compile/highlights + the chunked-upload endpoints, which exist but are unwired in app.js); the jobs table + max_workers=1 self-scheduling drain (job_worker.py) as the control plane; the per-rally doc docs/PER_RALLY_SELECTION.md (embed its RallyPicker + reuse its verified engine-contract anchors S3/S4 verbatim, adopt its honesty discipline); the khelsutra web/ Vite+React scaffold + its useRallySelection()/typed-API-client seams (P1-P2); the host-portability adapter shape (handleRequest(request,store) over

diStore|sqliteStore) as precedent for the one-URI storage contract; HOSTING_PLAN.md
 Access+Tunnel topology as the ready-made gate; the parity.yml/test.yml green-CI-gates-merge
 discipline; docs/PROJECT_DOC_TEMPLATE.md for the tracked doc. NET-NEW: the upload client wiring
 in the SPA; the Settings (compute+storage) screen reading GET /api/config; GET /api/videos
 (+owner scoping), GET /api/jobs (list/history), list-compiled-reels, and a review-playback
 stream endpoint; and the deferred UI<->bucket-worker bridge (an endpoint that enqueues 'python
 -m backend.worker' against a bucket and reflects outputs/<video_id>/ state into the same
 jobs/poll contract) plus the M3.5 NativeWasbRunner for real co-located GPU.",

"mvp_scope": [
 "Single box, storage.backend='local' (default), segmenter='gemini' (config.json default)
 OR stub+skip_ai_handoff for a key-free CPU run - the validated droplet path; NO new engine code
 for the core loop.",
 "Wire the EXISTING chunked-upload endpoints (/api/upload/init|part|complete) into the web/
 SPA file-picker, then feed the returned absolute path into POST /api/process (the one true
 client gap blocking browser upload).",
 "Drive the loop end-to-end: upload -> POST /api/process (202+job_id) -> poll GET
 /api/jobs/{id} with coarse stage text + a 'one job at a time' notice -> embed RallyPicker (POST
 /api/query -> segment_ids) -> POST /api/compile -> GET /api/highlights download. (S1-S4.)",
 "Settings screen: render GET /api/config to show the segmenter registry, default,
 detector_impl, and whether a GPU/Gemini key is present; allow choosing segmenter + toggling
 skip_ai_handoff per /api/process request. (S5 - compute placement is a visible setting even
 before remote GPU is wired.)",
 "Serve behind the HOSTING_PLAN Access+Tunnel gate; complete the go-live hardening (CORS
 narrow, allowed_video_root, rate limit/quota, rotate Gemini key) BEFORE any public exposure.",
 "Ship as a tracked doc in khelsutra (peer-of PER_RALLY_SELECTION.md, PROJECT_DOC_TEMPLATE
 structure, indexed in docs/README.md + a NEXT_STEPS pointer), one small PR per S-row off
 origin/master, no stacking, green CI gates merge."
],

"pros": [
 "Fastest real CUJ: the core loop already exists in FastAPI; the dominant net-new work is
 client-side upload wiring + reusing the approved RallyPicker, so a single box delivers
 upload->reel without new engine code.",
 "Honest 'compute is a setting': GET /api/config already exposes the
 registry/default/detector_impl, so Settings surfaces gemini/stub/native truthfully and the SAME
 UI later points at a bucket worker - the BYO-compute story the request-access form probes, with
 no UI rewrite.",
 "Reconciles cleanly with the decoupled model: storage=local is the degenerate
 StorageBackend config and the in-process worker is the degenerate ComputeBackend='local', so
 nothing fights the StorageBackend/worker design - the single box is literally the collapsed
 three-plane split.",
 "Privacy-forward default: stub+skip_ai_handoff runs with NO key and NO bytes to Google;
 gemini is an explicit opt-in toggle, matching the L0->L2 locality ladder and consent
 guardrails.",
 "Operationally proven: the exact box (DO droplet) already co-hosts site+engine and ran the
 full train+infer e2e against real Spaces with the GPU mocked - this design is the UI on top of a
 verified substrate."
],

"cons": [
 "max_workers=1 ceiling: strictly one job at a time; a 2nd submit queues with no
 position/ETA, and a server restart fails in-flight jobs - the UI must communicate this, it
 cannot hide it.",
 "No real GPU on the CPU droplet: real (non-mock) detection there is impossible; you get
 gemini (cloud, costs + leaks chunks) or stub (mock, not real rallies). Real local GPU =
 Windows+WSL (auto) today; a Linux GPU box needs the unbuilt M3.5 NativeWasbRunner.",
 "The decoupled bucket-worker path is invisible to the UI: driving 'python -m
 backend.worker' from the console (placement C) needs a net-new orchestration bridge that does
 not exist - so 'remote GPU as a setting' is design, not shipped.",
 "Engine has zero auth + CORS '*': the appliance is only as safe as the Cloudflare Access
 edge; a misconfigured/bypassed gate exposes everything, including unauthenticated uploads.",
 "Review playback + retention gaps: there is no endpoint to stream the source/proxy for in-
 UI rally review (S3 can list+select but not play-the-span), and uploaded originals are never
 deleted today - both must be addressed before broad exposure."
],

"honest_caveats": [
 "[verified] The UI working loop runs through the in-process FastAPI worker (jobs table

stores a local video_path); the decoupled 'python -m backend.worker {infer|train}' is a separate one-shot CLI the FastAPI does NOT invoke. 'One UI for both' is true for storage=local + the visible segmenter setting; the bucket-worker bridge is net-new and out of MVP.",

"[verified] detector_impl='native' REFUSES today - base _build_native_runner raises NotImplementedError (trajectory_hybrid.py:57-63); native_wasb_runner.py is the M3.5 next build. Co-located real GPU on Linux is NOT available until it lands; on Windows the WSL 'auto' path works.",

"[verified] The CPU droplet has NO GPU; the only GPU-free real segmenter is 'motion'. wasb/tracknet/fusion need a GPU+WSL/native; 'gemini' needs GEMINI_API_KEY and uploads <=1280px chunks to Google even in otherwise-local modes - must be in consent copy.",

"[verified] Engine FastAPI = no auth, CORS '*'; Cloudflare Access (edge) is the ONLY tenancy boundary in alpha. Per-user scoping is an unbuilt owner_email column; multi-tenant identity/RLS is P1.",

"[verified] Client gaps in app.js: the chunked-upload endpoints exist but are never called; there is no GET /api/videos, no GET /api/jobs list, no list-reels, no review-playback stream, no cancel; progress is coarse free-text on a 3s poll (no %/ETA/SSE). The console must add these as net-new (engine endpoints for the list/playback ones) or stub them honestly.",

"[verified] Uploaded originals under security.upload_dir are never deleted (retention is an open gap); a publicly-reachable single box accumulates many users' video - default to keeping proxies+intervals and add retention before broad exposure.",

"[design] This is a design only; nothing here is implemented. It should be written up as a tracked doc (peer-of PER_RALLY_SELECTION.md / REQUEST_ACCESS_FORM.md) with a S7 progress tracker as the source of truth, every claim tagged [verified]/[researched]/[design], and is owner-gated before any build."

```
]
}
]
```

Tasks:

1. Score each approach 1-5 on fit_to_arch (reconciles with the decoupled StorageBackend/worker model), exposure_safety (protects the no-auth engine on a public box), mvp_shippability (drives the full CUJ on the CPU droplet), byo_compute (one UI handles CPU/gemini + local-GPU + remote-GPU). Sum.
2. Recommend an architecture ? likely a HYBRID (e.g. unified reverse-proxied origin with edge auth, UI driving the FastAPI in-process worker for the single-box MVP, with the decoupled bucket+remote-GPU-worker as the BYO-compute scale path; gemini/stub on the CPU droplet, native WASB on a GPU box). Justify with scores.
3. Produce a topology_diagram (text) and a compute_placement_matrix.
4. Define the CUJs (upload->index->reel at minimum; plus re-open a past match, pick-by-query, settings/compute) and a FIRST-CLASS live_test_plan: how each CUJ is replayed live during development (mock engine for fast hermetic replay + real local engine for e2e; preview/browser-driven UI replay; engine-contract checks against a running FastAPI; automated CI regression; and the heaviest GPU/real-4K/golden-corpus E2E handed to the owner). This section is REQUIRED and must be concrete.
5. Crisp MVP + phased milestones.
6. engine_changes: concrete, each tagged new/exists/modify AND flagged as engine-repo changes to COORDINATE with the active decoupled/worker sessions (do not assume we write them unilaterally).
7. open_decisions (owner) + risks.
8. design_markdown: the full tracked doc body per the template, with S9 = CUJs & live-test plan. Honest about CPU-droplet (no local WASB -> gemini/stub), the no-auth API (edge auth required before public exposure), and shared-testbed boundaries. Cite grounding refs inline. Return the schema.

2. assistant (2026-06-20T09:14:04.958Z)

I'll analyze the three approaches and produce the consolidated recommendation. Let me first verify a few key engine facts before scoring.

3. user (2026-06-20T09:14:06.381Z)

```
625         if paths:
626             print(f"[REPORT] Technical report : {paths.get('technical_md')}")
627             print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
628             print(f"[REPORT] Machine JSON      : {paths.get('json')}")
629         return
```

```

630
631     # Start Local Server Mode
632     if args.server or not args.video_path:
633         print(f"[SERVER] Starting local FastAPI server on
http://localhost:{args.port}...")
634         # Resume self-scheduled work orphaned by the restart (EXECUTION_POLICY.md P3):
the
635         # in-process wake-timer for any parked 'waiting' job died with the old process,
so
636         # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
637         _get_executor().submit(_drain_due_jobs)
638         uvicorn.run(app, host="127.0.0.1", port=args.port)
639
640 if __name__ == "__main__":
641     main()
642

```

4. user (2026-06-20T09:14:06.956Z)

```

1     """Linux-native WASB shuttle-trajectory runner (no WSL).
2
3     The de-risk goal the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
4     "Optional Linux-native path"; ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md``
M3.5).
5     It is the sibling of :class:`~backend.pipeline.detectors.wasb_runner.WasbRunner` for a
6     Linux GPU box (GCP / a SaaS worker): it runs the same verified ``wasb_infer.py``
core
7     that the WSL adapter runs, so the trajectory CSV is byte-identical ? but WITHOUT the WSL
8     transport:
9
10     * NO ``WslCommandMixin`` (no ``wsl -d <distro> bash -c``), NO ``conda activate`` (the
11     ``wasb`` env's python is invoked by path), NO ``/mnt/c`` path translation
12     (``to_wsl_mnt_path``). Paths are native; the video is already local on the box.
13     * ``device`` is ``cuda`` (default ? the WSL parity path), ``mps`` or ``cpu``; threaded
14     into ``wasb_infer.py --device`` (and so into the Hydra ``runner.device`` override).
15     * weights / repo / python / scratch come from the environment first
16     (``WASB_WEIGHTS`` / ``WASB_REPO_DIR`` / ``WASB_PYTHON`` / ``WASB_SCRATCH``), then
17     ``indexing.wasb.*`` config, then a default ? so a box needs no config edits.
18
19     It emits the identical ``Frame,Visibility,X,Y`` CSV (same ``{stem}__{vid12}_wasb.csv``
20     name as the WSL runner) and reuses the model-agnostic ``parse_trajectory_csv`` /
21     ``trajectory_to_action_windows`` from ``tracknet_runner``, so the trajectory hybrids and
22     the windowing brain are unchanged. Selected via the existing ``detector_impl`` seam
23     (``trajectory_hybrid._resolve_runner`` ? ``_build_native_runner``);
``detector_impl="native"``.
24
25     Acceptance gate (owner / GPU box): byte-parity ? the SAME clip through the WSL
runner
26     (Windows) and this runner (Linux, ``device=cuda``) yields an identical trajectory CSV
and
27     identical candidate windows. That parity is a hardware check (a real GPU + the ``wasb``
28     env), so it is verified by the owner on the box ? not in this offline, subprocess-mocked
29     test suite.
30     """
31     import os
32     import shutil
33     import logging
34     import subprocess
35     from dataclasses import dataclass
36     from typing import Any, Dict, List, Optional, Tuple
37
38     from backend.pipeline.detectors.base import DetectorRunner
39     # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical to
WSL/TrackNet.
40     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner

```

```

41
42     logger = logging.getLogger(__name__)
43
44     # Path to the inference core we copy into the WASB repo's src/ (so it can import WASB's
45     # detectors/trackers/dataloaders + find configs/), exactly as the WSL adapter does.
46     _INFER_PY = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
47
48
49     @dataclass
50     class NativeWasbConfig:
51         """Resolved settings for a Linux-native WASB run.
52
53         Built via :meth:`from_indexing_cfg` with **environment overrides** taking precedence
54         over ``indexing.wasb.*`` config (the box sets env; no config edits needed).
55         ``device``
56         defaults to ``cuda`` (the WSL byte-parity path).
57         """
58         repo_dir: str = "~/models/WASB-SBDT"      # native clone of nttcom/WASB-SBDT
59         python_bin: str = "python"                # the `wasb` conda env python (invoked by
60         path, no activate)
61         weights_path: str = ""                    # REQUIRED (env WASB_WEIGHTS or
62         indexing.wasb.weights_path)
63         sport: str = "badminton"

```

5. assistant (2026-06-20T09:14:12.758Z)

I have enough verified context. The key facts are confirmed: uvicorn binds 127.0.0.1:8000 (main.py:638), and native_wasb_runner.py exists on this branch with a byte-parity acceptance gate owner-verified on a GPU box. Now I'll compose the consolidated recommendation.

6. user (2026-06-20T09:18:16.056Z)

Structured output provided successfully

7. assistant (2026-06-20T09:18:24.408Z)

Consolidated recommendation delivered via StructuredOutput.

Summary: I scored the three approaches (A=15, B=18, C=18) and recommended a **hybrid** of B's reverse-proxied unified-origin spine + C's "compute is a setting / bucket-worker as the reserved scale path" framing. The decisive factor is exposure-safety as a **structural** property ? verified at `backend/main.py:638` that the engine hard-binds `127.0.0.1:8000`, so combined with a firewall blocking 8000 and an edge gate (Caddy basicauth or cloudflared+Cloudflare Access), the no-auth/CORS-`` engine (`main.py:42`) is unreachable except through the proxy.

I also verified `native\_wasb\_runner.py` is present on this branch with an owner-only byte-parity acceptance gate, confirming native WASB is genuinely in-progress (M3.5) and not droplet-runnable ? which anchors the honest CPU-droplet caveat (gemini=real-cloud or stub=mock only).

The output includes all required pieces: scores, recommended architecture, topology diagram, compute placement matrix, four CUJs, a first-class five-layer live-test plan, MVP definition, six phased milestones, engine-changes flagged for coordination with the active decoupled/worker sessions, open decisions, risks, the full tracked-doc body (\$9 = CUJs & live-test plan), and grounding refs.